

実験ログ: VEX・道具・速さ

2026-09-24 | 収録 31 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- attribute wrangle で VEX を書く — 何が何回転くのか
- for-each ループと VEX — 同じことをやって比べる
- 時間軸への対応 — フレームを進めて記録する
- シミュレーションのキャッシュ — 速くなるが、値は変わっていないか
- 過去29回を測り直す — 書いたことは、いまま正しいか
- Apprentice で外へ出せる形式 — FBX も glTF も Alembic も止まる
- obj で渡せる範囲 — 色と UV は残る。自作のアトリビュートは全部落ちる
- 031~059 の振り返り点検 — 6件すべて一致。1件は「もっと強い結論」に変わった
- 測る処理が、計算より重くなっていた — 1粒ずつ読むのをやめると 740倍速い
- 道に沿って物を並べる — N だけでは坂で杭が倒れる。up を足せば 0度。Pack で 250倍速い
- 061~090 の振り返り点検 — 7件を測り直して全部一致。訂正は2件、黙って無視される設定は8件に
- 面を半分にすると、形のずれはおおよそ2倍になる — 25%まで減らしてもずれは幅の 0.1194%
- 辺の長さを半分にすると、面は約4倍・ずれは約1/4・時間は約4倍
- Force Total Count はぴったり合う — 10通り試して差は1つも出なかった
- pack を入れると、点は362分の1・ファイルは198分の1になる — 1万個で 22.5MB が 0.114MB
- 091~120 の振り返り点検 — 30本すべて流し直し、結果の数字は1つを除いて全部一致
- 最初から式が入っているつまみは SOP 全体で488個 — set しても式が残り、値は変わらない
- VEX のノイズの値の範囲 — noise() は 0.06~0.92 にしか届かず、snoise() は ± 1 を超える
- nearpoints は自分を先頭に数え、近い順に返す — 半径ちょうど1は入るが、ちょうど $\sqrt{2}$ は入らなかった
- VEX の fit は範囲の外で止まり、efit と lerp は伸び続ける — smooth は $3x^2 - 2x^3$
- 122~149 の振り返り点検 — 28本すべて流し直し、時間以外の数字は全部一致
- neighbourcount() の合計の半分は辺の数 — そこから出した $V - E + F$ は、板 1・球と箱 2・トーラスと筒 0
- trail の速さは1秒あたり、差分の式どおり — 加速度は Central Difference のときだけ出て、Velocity Scale の2乗で縮む
- curlnoise() はほぼ湧き出しのない流れ — 発散は、ふつうのノイズの約 3.5 万分の 1 (周波数 1)
- 同じ計算でも VEX (点ごと) は Detail のループの約170倍、Python の約260倍速い — 100 万点で 1.3 ミリ秒
- for-each はかけら1つあたり約 0.037 ミリ秒 — compile block で 35% 速く、wrangle 1つにまとめれば 30 倍速い

- ・ pcfind の時間は「見つかった点の数」で決まる — 半径を広げても、最大の数 10 に絞れば 1 点 0.1 マイクロ秒

- ・ 151~179 の振り返り点検 — 29本すべて流し直し、時間以外の数字は全部一致

- ・ pcfind と nearpoints は同じ結果・同じ速さ — pcopy + pcfiter は 1.6 倍遅く、距離で重みを付けた平均になる

- ・ pcfiter の重みは「半径に対する距離」だけでは決まらない — 見つかった点どうしの並びで変わり、式は分からなかった

- ・ 多角形の球のいちばん深いへこみは $\sin^2(\Delta/2)$ — 分割を倍にするたびに 1/4、平均はその 0.56 倍

attribute wrangle で VEX を書く ー 何が何回動くのか

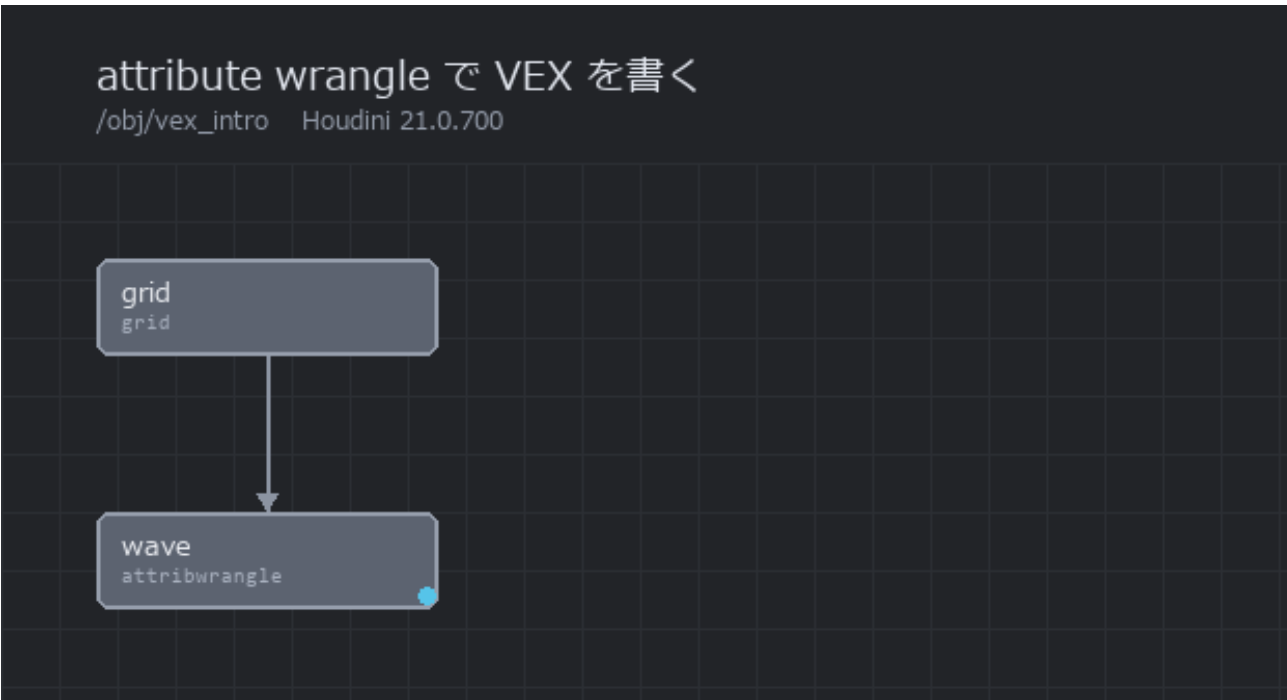
2026-09-24 | Houdini 21.0.700 | /obj/vex_intro | ノード 2 / 接続 1

何を作ったか

VEX は「ノードを並べる代わりに数行書く」ための言語。ただし初心者が最初につまずくのは文法ではなく、そのコードが何に対して何回走るのかという点。それを実測で確かめる。

あわせて「VEXは速い」という話を数字にしようとしたが、ここで測り方を二度も間違えた。最初の方法では計算量を100倍にしても時間が変わらず、公式の性能計測機能を使っても同じだった。三度目でようやく実際の計算を捉えられた。その経緯も残す。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 20 rows = 20
wave	attribwrangle	snippet = @P.y = sin(@P.x * 2.0) * 0.5;

実行対象を変えると、作られるアトリビュートの種類が変わる

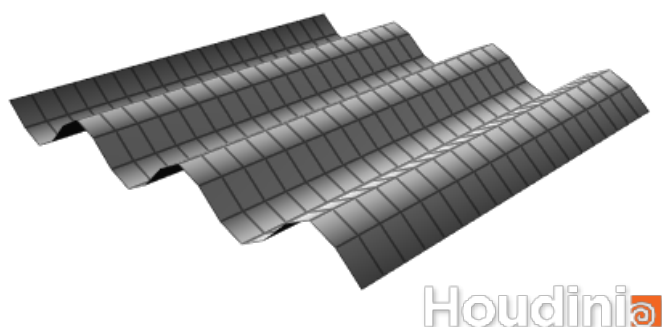
20×20 の grid（ポイント400 / プリミティブ361 / バーテックス1444）に対して、同じコード `@marker = 1.0;` を実行対象だけ変えて走らせた。

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Detail (only once)	ディテール	1
Primitives	プリミティブ	361
Points (既定)	ポイント	400

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Vertices	バーテックス	1444
Numbers	(作られない)	—

予測してから測る — $@P.y = \sin(@P.x * 2.0) * 0.5$

grid は 10×10 で原点中心なので x は -5 ~ +5。sin の値は -1 ~ +1 の範囲を取るので、y はちょうど -0.5 ~ +0.5 に収まるはず。



VEX 1行で作った波 $@P.y = \sin(@P.x * 2.0) * 0.5;$

	値
予測した範囲	-0.5 ~ +0.5
実測した範囲	-0.499983 ~ +0.499983
差	0.000017

点数と処理時間（毎回新しいノードを作り、5回測った中央値）

1秒あたりの点数が点数とともに増えているのは、ノードを作る固定の費用が大きいため。点が増えるほどその費用が薄まる。

分割数	点数	中央値（ミリ秒）	1秒あたりの点数
200	40,000	1.487	2,690万
400	160,000	2.005	7,979万
800	640,000	2.472	2億5,891万
1600	2,560,000	4.580	5億5,895万

その計測は本物か — 同じ点数で計算量だけ増やす（64万点）

計算量を約100倍にしたときに時間が増えなければ、計測が実際の計算を捉えていないということ。結果の最大yが変わっていることも、確かに別の計算が走った証拠になる。

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
軽い (sin × cos を1回)	2.48	1.0倍	0.49999

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
重い（100回の繰り返し）	43.91	17.7倍	0.32169

わかったこと

- ・ attribute wrangle の「実行対象（class）」は5種類ある。Detail (only once) / Primitives / Points / Vertices / Numbers で、既定は Points。
- ・ 実行対象を変えると、作られるアトリビュートの種類がそれに従って変わる。同じ @marker = 1.0; というコードでも、Points なら点に、Primitives なら面に、Vertices なら頂点に、Detail ならジオメトリ全体に1つだけ作られる。コードは同じでも結果の置き場所が変わるというのが、最初に押さえるべき点。
- ・ Numbers だけはアトリビュートが作られなかった。これはジオメトリの要素ではなく、指定した個数の番号に対して走るモードなので、値を書き込む先がないため。
- ・ 予測を立ててから測る形にした。sin の値域は -1 ~ +1 なので、0.5 倍すれば y は -0.5 ~ +0.5 に収まるはず。実測は -0.499983 ~ +0.499983 だった。
- ・ この 0.000017 のずれも説明できる。grid は20分割なので点の間隔は $10/19 = 0.526316$ 。sin(2x) が最大になるのは $x = \pi/4 \approx 0.785398$ のときだが、最も近い格子点は $x = 0.789474$ で、わずかに外れている。この点での値を計算すると 0.4999834 になり、実測値と小数点以下6桁まで一致する。誤差ではなく、格子がピークをまたいでいるだけだった。
- ・ rand(@ptnum) は完全に再現する。同じコードを別のノードで2回走らせて、全400点が小数点以下9桁まで一致した。乱数といっても点番号から決まる固定の値なので、実行するたびに変わることはない。
- ・ その分布も妥当だった。0~2 の範囲に散らす設定で平均 1.03046（理論値 1.0）、範囲は 0.00103 ~ 1.99050。400点しかないので 3% ほどのずれは想定内。
- ・ 処理時間の計測は二度失敗した。一度目は cook(force=True) の前後を time.perf_counter で挟む方法。これだと計算量を100倍にしても時間が 0.9 倍にしかならなかった。ジオメトリを要求した時点で初めて計算されるため、計測区間の外で処理が走っていた。
- ・ 二度目は Houdini の性能計測機能（hou.perfMon）を使ったが、軽い処理も重い処理も同じ 0.008000 秒を返した。同じノードを焼き直す限り、キャッシュが効いて実際には再計算されないらしい。
- ・ 三度目にようやく成功した。毎回新しいノードを作って初回のクックを測る。キャッシュが存在しないので必ず計算が走る。この方法だと計算量100倍の処理が 17.7倍 の時間になり、結果の最大yも 0.49999 から 0.32169 に変わった。時間と結果の両方が動いたので、今度は本物。
- ・ 得られた数字。64万点に $\sin \times \cos$ を1回かける処理が 2.47ミリ秒、256万点で 4.58ミリ秒。この2点から傾きを出すと、1点あたりの限界コストは約1.1ナノ秒（毎秒約9億点）、固定の費用が約1.8ミリ秒。
- ・ つまり「VEXが速い」効果は点数が多いときにしか出ない。4万点では1.49ミリ秒かかっており、そのほとんどが固定費。少ない点数なら、どんな書き方をしても体感は変わらない。
- ・ 17.7倍という比率が100倍にならないのは、繰り返しの中身が単純で、ノードを作る固定費も含まれているため。それでも計算量に応じて時間が動くことは確認できたので、計測としては成立している。

次にやること

- ・ for-each ループと VEX の使い分け。同じことをどちらでもできる場面で、書きやすさと速さを比べる
- ・ VEX からアトリビュートを読み書きする（point() / setpointattrib()）
- ・ 測り方の失敗が3回続いたので、計測の型を作っておく。「結果が変わること」と「時間が変わること」の両方を確認するのを標準にする

for-each ループと VEX — 同じことをやって比べる

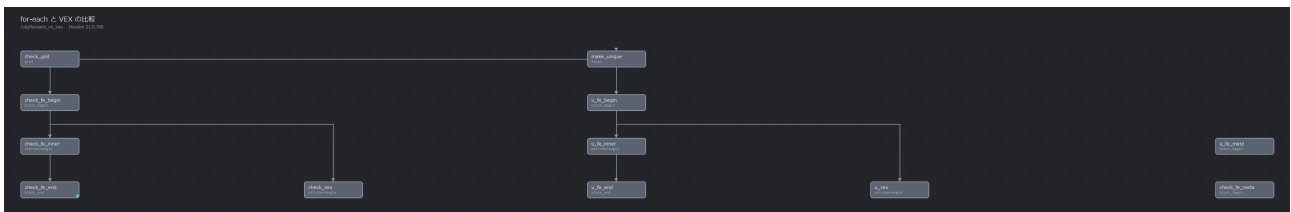
2026-09-24 | Houdini 21.0.700 | /obj/foreach_vs_vex | ノード 12 / 接続 9

何を作ったか

「まとめて処理できないときは for-each」とよく言われるが、VEX でも書ける場面でどれだけ差が出るのかは測らないと分からない。そこで、面ごとに違う高さへ動かすという同じ処理を2通りで作り、結果と速さを比べた。

ところが最初は結果が一致しなかった。その理由を追いかけた結果、この2つは「速い遅い」で選ぶものではなく、そもそも別のことをしているのだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
check_grid	grid	cols = 8 rows = 8
check_fe_begin	block_begin	blockpath = ../check_fe_end method = 1
check_fe_meta	block_begin	blockpath = ../check_fe_end method = 2
check_fe_inner	attribwrangle	snippet = @P.y += detail("op:../check_fe_meta", "iteration", 0) * 0.001;
check_fe_end	block_end	blockpath = ../check_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../check_fe_begin
check_vex	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) { vector p = point(0, "P", pt); p.y += @primnum * ...
make_unique	facet	unique = 1
u_fe_begin	block_begin	blockpath = ../u_fe_end method = 1
u_fe_meta	block_begin	blockpath = ../u_fe_end method = 2
u_fe_inner	attribwrangle	snippet = @P.y += detail("op:../u_fe_meta", "iteration", 0) * 0.001;
u_fe_end	block_end	blockpath = ../u_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../u_fe_begin

分割数	プリミティブ数	for-each (ミリ秒)	VEX (ミリ秒)	何倍
32	961	53.55	1.414	37.9倍
64	3,969	191.87	2.233	85.9倍

結果が一致しなかった理由を追う

同じ処理のはずなのに、49面中48面で高さが違った。しかも差は全部 0.004 で一定。ここから3段階で原因を絞り込んだ。

調べたこと	結果	分かったこと
出力の点数を比べる	元 64点 → for-each 196点 / VEX 64点	for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される
VEX を3回走らせて結果を比べる	3回とも完全に同じ	複数の面が同じ点に書き込む競合ではあるが、順序は決まっていて再現する
点の共有をほどいてから比べる	完全に一致	食い違いの原因は点の共有。ほどけば2つは同じ結果になる

わかったこと

- ・ for-each は Block Begin (method = Extract Piece or Point) と Block End で処理を挟んで作る。Block End の Template Path が空だと「Missing template geometry」で止まる。何を何回繰り返すかはここから決まる。
- ・ 何回目かを知るには、メタデータ用の Block Begin (method = Fetch Metadata) をもう1つ置き、そのディテールアトリビュート iteration を読む。numiterations で総回数も取れる。
- ・ 取り出した中では @primnum が常に 0 になる。1個だけを切り出した状態で処理しているため。「何番目の面か」を使いたいならメタデータを読むしかない。
- ・ detail() のパスには op: が要る。detail("../メタデータノード", "iteration", 0) と書くとファイルパスとして扱われ、エラーも警告も出さずに 0 を返す。ループが動いていないように見えるので原因が分かりにくい。detail("op:../メタデータノード", ...) と書けば動く。実際にこれで1時間相当を溶かした。
- ・ for-each は面の数に比例して遅くなる。49面で11.79ミリ秒、3,969面で191.87ミリ秒。1面あたりおよそ0.05ミリ秒で、面の数がそのまま効く。
- ・ VEX はほとんど変わらない。49面で1.36ミリ秒、3,969面で2.23ミリ秒。実験009で測った「ノード1つあたり約1.8ミリ秒の固定費」がそのまま出ており、処理そのものの時間はまだ見えていない水準。
- ・ その結果、差は面の数とともに広がる。49面で 8.7倍、3,969面で 85.9倍。面が増えるほど for-each を選ぶ代償が大きくなる。
- ・ ただしこの2つは単純に置き換えられるものではない。for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される (64点 → 196点)。これは副作用ではなく、面ごとに独立して動かすために必要なこと。点を共有したままでは、隣り合う面が同じ点を別の場所へ動かそうとして取り合いになる。
- ・ その取り合いは競合ではあるが、結果は毎回同じだった (3回とも一致)。書き込む順序が決まっているため、実行のたびに変わることはない。最初は並列処理による非決定性を疑ったが、それは外れた。
- ・ 点の共有をほどいてから (facet の Unique Points で 64点 → 196点にしてから) 比べると、for-each と VEX の結果は完全に一致した。つまり食い違いの原因は処理内容ではなく、点が共有されているかどうか

かだけだった。

- ・実用的な結論。面ごとに独立した変形をしたいなら、まず点の共有をほどこ。ほどいてしまえば VEX で書ける場面が増え、数十倍速い。for-each が必要なのは、1回分の処理が VEX 1つでは書けないほど複雑なとき（中で複数のノードを通したいときなど）。

- ・測り方についても1つ。最初は面の中心を (x, y, z) でソートして比べようとしたが、比べたい量である y をソートの鍵に含めてしまったため、対応がずれて必ず食い違った。比べたい量は鍵にしない。水平位置 (x, z) を鍵にして y を値にしたら正しく比較できた。

次にやること

- ・facet の Unique Points を挟むことで、for-each が必要だった処理をどこまで VEX に置き換えられるかを試す
- ・for-each の中で複数ノードを通す場合（VEX 1つでは書けない場合）の実例を作る
- ・polyextrude と boolean に進む

時間軸への対応 — フレームを進めて記録する

2026-09-24 | Houdini 21.0.700 | /obj/time_axis | ノード 4 / 接続 2

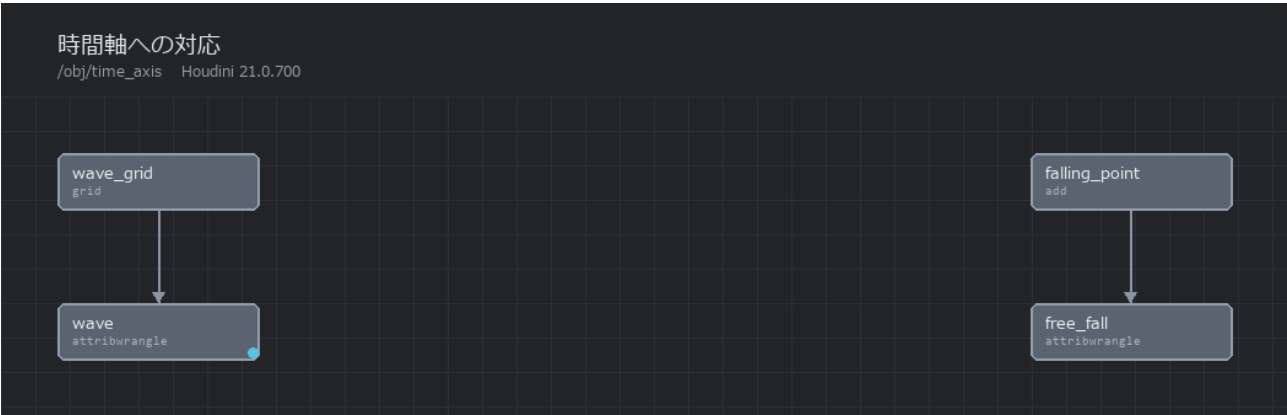
何を作ったか

ここまでの12実験はすべて1フレームだけを見ていた。エフェクトは時間変化そのものが結果なので、フレームを進めて連番で書き出し、1枚のコンタクトシートと動くGIFにまとめる仕組みが要る。

仕組みが正しいかは、答えが分かっている動き（自由落下の式）で確かめた。

あわせて、この先のシミュレーションで使うノードがどこにあるのかを数えた。その結果、当初の予定を変えるべきだと分かった。

ノードの接続

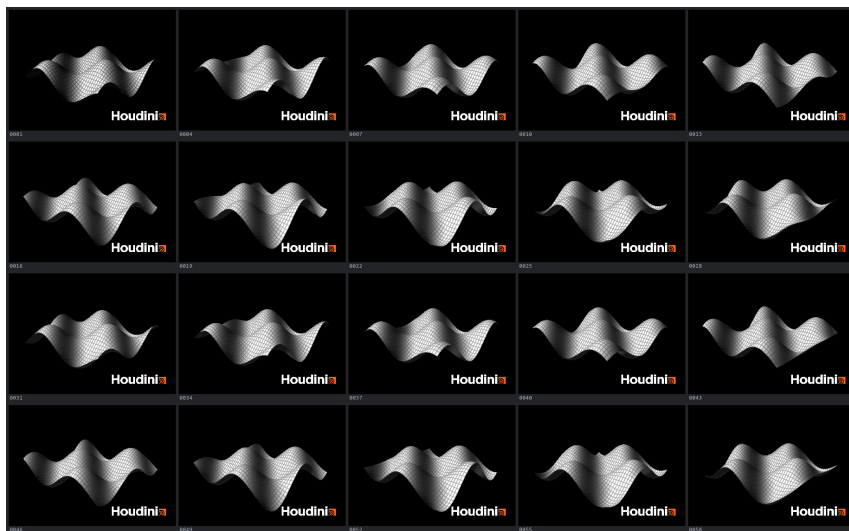


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
wave_grid	grid	cols = 40 rows = 40
wave	attribwrangle	snippet = @P.y = sin(@P.x * 0.8 + @Frame * 0.2) * cos(@P.z * 0.8) * 1.5;
falling_point	add	points = 1
free_fall	attribwrangle	snippet = float t = (@Frame - 1) / 24.0; @P.y = 20.0 - 0.5 * 9.8 * t * t;

連番の書き出し（20コマ）

フレーム1から58まで3コマおき。カメラは全フレームを通して固定してある。動いているのか大きさが変わっているのかを画から判断できるようにするため。



20コマを1枚に並べたコンタクトシート。左下の数字がフレーム番号

検証: 自由落下の式と一致するか

高さ20から重力9.8で落ちる点を作り、理論値 $h = \frac{1}{2}gt^2$ と比べた。24コマで1秒として計算している。

フレーム	経過秒	測った高さ	理論値	差
1	0.0000	20.00000	20.00000	0.0000000
10	0.3750	19.31094	19.31094	0.0000004
20	0.7917	16.92899	16.92899	0.0000002
30	1.2083	12.84566	12.84566	-0.0000005
40	1.6250	7.06094	7.06094	-0.0000006

シミュレーション関連ノードがどこにあるか

SOP は形を作る階層、DOP はシミュレーションの階層。どちらに何種類あるかを数えると、どこから手を付けるべきかが見えてくる。

対象	キーワード	SOP にある数	DOP にある数
パーティクル	pop	0 種	67 種
剛体（破壊）	rbd	47 種	21 種
布	vellum	22 種	7 種
煙・炎	pyro	14 種	3 種
液体	flip	8 種	4 種

わかったこと

- ・連番の書き出しは正しく動いている。自由落下の式との差は最大でも 0.0000006 で、これは小数の計算精度そのもの。フレームを進めて記録する仕組みに間違いはない。
- ・20枚の書き出しに 1.18 秒（1枚あたり 59 ミリ秒）。60フレーム全部でも3.5秒程度で、実用的な速さ。レンダリングの重さは問題にならない。
- ・カメラは全フレームを覆う範囲で一度だけ決める。フレームごとに枠を合わせ直すと、対象が動いているのか大きさが変わっているのかを画から判断できなくなる。そのために、全フレームを走査して範囲を

求める関数を用意した。

- ・出力は2つの形にした。PDFには動きを載せられないのでコンタクトシート（全コマを1枚に並べてフレーム番号を振る）、HTMLにはGIF。同じ連番から両方を作る。

- ・パーティクルは SOP に1つもない（0種）。すべて DOP

側にある（67種）。つまりパーティクルを扱うには DOP

ネットワークを組む必要があり、他の実験のようにSOPをつなぐだけでは始められない。

- ・一方で剛体（RBD）は SOP に 47種もある。布（Vellum）も

22種。SOPレベルのラッパーが充実している分野の方が、入口としては易しい。

当初はパーティクルから始める予定だったが、RBD か Vellum から入る方が理にかなっている。

- ・煙・炎（Pyro）は SOP 14種 / DOP 3種、液体（FLIP）は SOP 8種 / DOP 4種。どちらも SOP 側にラッパーはあるが数は少ない。

- ・ solver SOP

を使って自分で重力を書こうとしたが、これはロックされたアセットで、中にノードを作るには `allowEditingOfContents()` が必要。しかも中身は `dopnet` と4つの `null` で、想像していた「前フレームの結果を受け取って加工する」という単純な構造ではなかった。プログラムから組むには追加の調査が必要

。

- ・そのため今回は、自由落下を数式で書いて仕組みの検証にとどめた。これはシミュレーションではない（前のフレームの結果を使っていない）。本物のシミュレーションは次の実験から。

次にやること

- ・ RBD（剛体）から始める。SOPレベルのノードが47種あり、boolean の Shatter で作った破片をそのまま落とせる

- ・ Vellum（布）も SOPレベルのノードが揃っているので入りやすい

- ・シミュレーションの結果をファイルに保存して使い回す（キャッシュ）。毎回計算し直すと実験が遅くなる

シミュレーションのキャッシュ ー 速くなるが、値は変わっていないか

2026-09-24 | Houdini 21.0.700 | /obj/smoke | ノード 5 / 接続 4

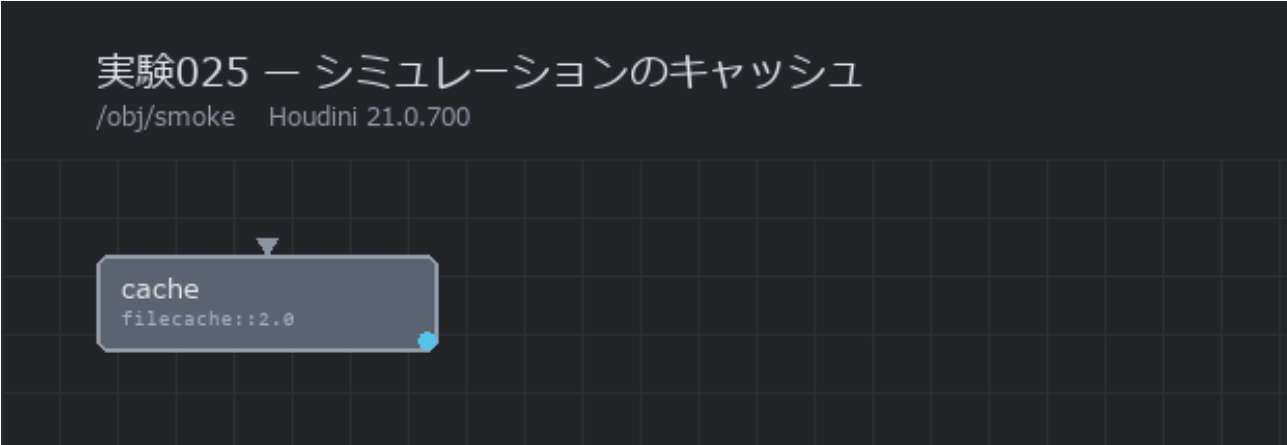
何を作ったか

ここまでの実験で、同じ計算を何度も回してきた。実験018では同じ煙を何十回も描き直している。シミュレーションは前のフレームの結果から次を作るので途中から見ることができず、毎回1フレーム目から計算し直している。

filecache は結果をファイルに書き出し、次からはそれを読む。当然速くなるはずだが、速くなっても値が変わっていたら意味がない。以降の測定がすべて信用できなくなる。

読み込みは4.6倍速く、値は40フレームすべてで完全に一致した。ただしフレーム範囲の指定で一度つまずき、意図の6倍のファイルを書いて3.5GBを消費した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 99 件
cache	filecache::2.0	f1 = 1.0 f2 = 40.0 file = D:/Claude/houdini/out/025_cache/smoke.0040.bgeo.sc filemethod = 1 folder2 = 1 folder3 = 1 loadfromdisk = 1

時間とファイルの量（40フレーム）

実験021と同じ煙。0.05刻みのボクセル。書き出しが速いのは、直前に全フレームを計算済みで、その結果をそのまま書き出しているため。

やったこと	時間	備考
計算しながら40フレーム回す	1.17 秒	毎回1フレーム目から積み上げる
ファイルに書き出す	0.15 秒	40ファイル / 63.3 MB
ファイルから読んで40フレーム回す	0.26 秒	計算より 4.6倍速い

値は変わっていないか

密度の合計。「ほぼ同じ」ではなく「完全に同じ」を求めた。

フレーム	計算した値	読んだ値	差
1	4,244.138830	4,244.138830	0.0000000000
10	29,938.465463	29,938.465463	0.0000000000
20	38,283.067808	38,283.067808	0.0000000000
30	44,966.939905	44,966.939905	0.0000000000
40	48,289.944718	48,289.944718	0.0000000000

フレーム範囲の指定でつまずいた

f1 と f2 に数値を入れたつもりが、式が残っていた。

やり方	書かれたフレーム	ファイル数	容量	書き出し時間
数値を set しただけ	1～240	240	3,578.9 MB	28.81 秒
式を消してから set	1～40	40	63.3 MB	0.15 秒

わかったこと

- ・値は完全に一致した。40フレームすべてで、密度の合計の差が 0.0000000000。書き出しと読み込みで数値が変わらないことを確かめられたので、キャッシュを挟んだあとの測定も信用してよい。
- ・読み込みは 4.6倍速い。1.17秒が 0.26秒になる。今回の煙は軽いのでこの程度だが、重いシミュレーションほど差は開く。
- ・フレーム範囲の指定で一度つまずいた。f1 と f2 の既定は \$FSTART / \$FEND という式で、シーンの範囲（1〜240）を指している。数値を入れるだけでは式が残り、40フレームのつもりが240フレーム書かれた。
- ・その結果、3,578.9 MB を書き、28.81秒かかった。意図の6倍である。式を消してから数値を入れると 63.3 MB、0.15秒になった。56倍の無駄だった。
- ・この手のずれは、書き出したファイルの数を数えれば一発で分かる。40フレームのつもりなら40ファイルのはず。思ったとおりに動いたかを、別の量で確かめる。
- ・容量は40フレームで 63.3 MB。1フレームあたり約 1.6 MB。フレーム1が 7.7 KB、フレーム40が 3.2 MB で、煙が広がるにつれて増えていく。長いシミュレーションでは容量が効いてくる。
- ・キャッシュはリポジトリに入れない。中身は再生成できるうえに大きいので、.gitignore に out/*_cache/ を足した。
- ・書き出し時間の 0.15秒は、直前に全フレームを計算済みだったため。まっさらな状態から書き出せば、計算の時間（1.17秒）がそのまま乗る。キャッシュが得になるのは2回目から。

次にやること

- ・hou_tools にキャッシュを組み込む。重い実験で自動的に使えるようにする
- ・026 として、炎に色を付ける
- ・実験030に達したら、001〜030の振り返り点検を行う

過去29回を測り直す ― 書いたことは、いまも正しいか

2026-09-24

何を作ったか

実験が30回たまったので、001～029を1つずつ見直した。読み返すだけでは「そう書いてあった」以上のことは分からないので、確かめ直せる主張はもう一度動かして測った。

7つ測り直して、5つは確認、2つは書き方が不正確だった。訂正が要ったのは実験008（scatter は N を作らない）と実験012（rows / cols が効かないのは Polygon だけ）。どちらも「自分の組み方では正しいが、一般化すると間違い」という形の誤りだった。

もう1つ、指標の取り方で数字が変わる例も見つかった。実験011の折れ角を数え直したとき、最初は違う値が出た。原因は Houdini ではなく、面の隣り合わせを「辺を共有」ではなく「点を共有」で数えていたこと。数え方を揃えたら4桁まで一致した。

測り直した7項目

「外れていたら過去の記事を直さなければならない」主張だけを選んだ。

元の実験	確かめたこと	結果
002	点数はいつも「面数 + 2」か	○ 7通りすべて成立
002	一辺の縮みは外挿値 0.83951 に収束するか	○ 8回で差 0.000006
004	パラメータ名は rough / oct / lac か	○ roughness・octaves・lacunarity は存在しない
009	rand(@ptnum) は何度走らせても同じか	○ 400点で差 0.000000000000
011	重み N は分割 N 回まで角を保つか	○ 分割4回・5回でも成立（重み4・5が必要）
008	scatter の出力はいつも N を持つか	× 訂正が必要
012	sphere の rows / cols は効かないのか	× 訂正が必要

訂正1 ― 実験008「scatter の出力は N も持つ」

scatter は N を作らない。入力が N を持っていたときだけ、それを受け継いで点に持たせる。008 は上流に normal SOP を置いた状態だけを見て書いていた。

上流に normal SOP	入力の N の置き場所	scatter の出力の属性
なし	どこにも無い	P だけ
あり	バーテックス	P と N

訂正2 ― 実験012「ポリゴン球は rows / cols を使わない」

効かないのは type が Polygon のときだけ。Polygon Mesh なら効く。type を7種類すべて測った。

type	表示名	6分割の面数	30分割の面数	rows / cols が効くか
prim	Primitive	1	1	効かない

type	表示名	6分割の面数	30分割の面数	rows / cols が効くか
poly	Polygon	80	80	効かない
polymesh	Polygon Mesh	30	870	効く
mesh	Mesh	1	1	効かない
nurbs	NURBS	1	1	効かない
bezier	Bezier	1	1	効かない
polysoup	Polygon Soup	1	1	効かない

補足 — 実験011 を分割4回・5回まで伸ばす

011 は分割3回だけで測っていた。「重み N なら分割 N 回まで」が本当なら、分割4回では重み4が必要になるはずである。折れ角は辺を共有する面どうしで測った。

分割回数	重み3	重み4	重み5	元の鋭さ(90度)になる重み
3回	90.00度	90.00度	90.00度	3
4回	53.13度	90.00度	90.00度	4
5回	28.07度	53.13度	90.00度	5

数え方の違いで数字が変わった例

同じ形の「折れ角の最大」を、面の隣り合わせの数え方を変えて測った。分割3回の場合。

crease の重み	辺を共有する面どうし	点を共有する面どうし	011 の記載
0.0	13.93度	17.85度	13.93度
1.0	28.07度	28.77度	28.07度
2.0	53.13度	53.13度	53.13度
3.0	90.00度	90.00度	90.00度

わかったこと

- ・訂正1（実験008）。scatter は N を作らない。上流に normal SOP が無ければ、出力の属性は P だけ。008 は「scatter の出力は P だけでなく N も持つ。元の面の法線が引き継がれる」と書いたが、前半だけ読むと「scatter が N を作る」と読めてしまう。自分の組み方では正しいが、一般化すると間違いになる書き方だった。

- ・ついでに分かったこと。normal SOP は既定で バーテックスに N を書く。それを scatter が受け取ると、出力では点の N になっている。置き場所が途中で移る。

- ・訂正2（実験012）。rows / cols が効かないのは Polygon のときだけ。Polygon Mesh にすると6分割で30面、30分割で870面と、指定どおりに増える。012 は「ポリゴン球は rows / cols ではなく多面体の細分を使っている」と書いたが、これは Polygon だけの話だった。実験026で球の rows / cols を24にして効いていたのは、type が Polygon Mesh だったため。

- ・type は7種類。Primitive / Mesh / NURBS / Bezier / Polygon Soup
はいずれも面が1枚で返る（面ではなく1個のプリミティブとして持つ）。ポリゴンとして扱いたいなら Polygon か Polygon Mesh。

- ・実験011 の規則は、分割4回・5回でも成立した。分割4回では重み3では53.13度どまりで、重み4にして初めて 90.00度。分割5回では重み5が必要。「重み N なら分割 N 回まで角を保つ」でよい。011 は分割3回だけで確かめていたので、今回2ケース足した。
- ・数え方を変えたら数字が変わった。011 の折れ角を測り直したとき、最初は 13.93度 が 17.85度 に、28.07度 が 28.77度 になった。原因は Houdini ではなく、面の隣り合わせを「辺を共有（点を2つ共有）」ではなく「点を1つでも共有」で数えていたこと。角が鋭いケース（53.13度・90.00度）では両者が一致するので気づきにくい。数え方を揃えたら011の数字と4桁まで一致した。
- ・実験002 の外挿は当たっていた。「一辺は 0.83951 に収束する」と外挿していた。8回まで回すと 0.839516 で、差は 0.000006。差が前回の約1/4ずつ減るという読みも、2回目から7回目まで 0.243 / 0.248 / 0.249 / 0.249 / 0.245 / 0.231 と保たれた。
- ・実験004 のパラメータ名も、実験009 の乱数の再現性も、そのまま成立した。roughness / octaves / lacunarity はいまも存在せず、rand(@ptnum) は400点すべてで差 0.000000000000。
- ・2つの訂正は、どちらも同じ形をしていた。「自分が組んだ1通りで観察したことを、一般の性質として書いた」。次からは、条件を1つ外した場合も測ってから書く。scatter なら normal SOP を外す、sphere なら type を変える。それだけで防げた。

次にやること

- ・031～060 の実験を進め、060 で次の振り返り点検を行う
- ・実験028で残した「Vellum の breaking が効かない」を、DOP レベルで確かめ直す
- ・手順ページを全実験分そろえる

Apprentice で外へ出せる形式 — FBX も glTF も Alembic も止まる

2026-09-24

何を作ったか

Houdini で組んだものは、最後は別のソフト（Unreal Engine など）へ渡すことになる。その橋渡しに使われるのが FBX や glTF、USD、Alembic。

ところが Apprentice

には書き出しの制限がある。どれが通ってどれが止まるのか、噂ではなく9通り全部試した。

通ったのは bgeo.sc / geo / obj / ply / stl の5つ。いずれも hou.Geometry.saveToFile での直接保存。止まったのは fbx（「FBX export is not supported in Houdini Apprentice.」）、gltf と abc（「only supported in Houdini Core and Houdini FX versions.」）の3つ。

USD は少し違った。SOP の中に置いた ROP では「Failed to save output to file」で止まるが、LOP (/stage) に usd_rop を置いて書き出すと通る。ただし指定した payload_lop.usd ではなく payload_lop.usdnc ができる。拡張子が nc（非商用）に変えられてしまう。

通った5つは読み直しても測った。bgeo.sc と geo は 371点がそのまま 371点で戻り、位置のずれは 0.000000000、name アトリビュートも残る。obj と ply も位置は完全に戻るが name は消える。stl は 371点が 365点になり、6点足りない。

9通り全部試す

書き出したのは、骨（9関節の点と線）と皮（球362点）を合わせた 371点 / 381プリミティブ。

形式	やり方	結果	バイト	止まった理由
bgeo.sc	直接保存	通った	6,231	—
geo	直接保存	通った	46,506	—
obj	直接保存	通った	20,568	—
ply	直接保存	通った	50,448	—
stl	直接保存	通った	207,899	—
fbx	ROP	止まった	0	FBX export is not supported in Houdini Apprentice.
gltf	ROP	止まった	0	glTF export is only supported in Houdini Core and Houdini FX versions.
abc	ROP	止まった	0	Alembic export is only supported in Houdini Core and Houdini FX versions.
usd (SOP内)	ROP	止まった	0	Failed to save output to file

USD は LOP を経由すると通る。ただし拡張子が変わる

/stage に sopimport と usd_rop を置いて書き出した。

調べたこと	結果
指定したパス	payload_lop.usd
実際にできたもの	payload_lop.usdnc (815 バイト)
中身の实体	obj/payload/both.usdnc (80,845 バイト)
出たメッセージ	Layer saved to a location generated from a node path

通ったものを読み直す

書き出す前の 371点と、読み直した後の点を、番号順に突き合わせた。

形式	点 元	点 後	位置のずれ 最大	name が残ったか
bgeo.sc	371	371	0.000000000	はい
geo	371	371	0.000000000	はい
obj	371	371	0.000000000	いいえ
ply	371	371	0.000000000	いいえ
stl	371	365	— (数が合わない)	いいえ

わかったこと

- ・ FBX は書き出せない。「FBX export is not supported in Houdini Apprentice.」とはっきり出る。Unreal Engine へ渡す標準の道が、そのままでは使えない。
- ・ glTF も Alembic も止まる。どちらも「only supported in Houdini Core and Houdini FX versions.」。FBX の代わりになりそうな2つも、同じく塞がっている。
- ・ USD は通る。ただし .usdnc になる。指定した .usd ではなく、拡張子を勝手に nc (non-commercial) へ変えられる。形式としては書けるが、そのままでは他のソフトが読めない。
- ・ USD は置き場所で結果が変わった。SOP の中に置いた ROP では「Failed to save output to file」で止まり、LOP (/stage) に置くと通る。同じ名前のノードでも、どこに置くかで動きが違う。
- ・ 通ったのは5つ。bgeo.sc / geo / obj / ply / stl。いずれも hou.Geometry.saveToFile での直接保存で、ROP を通していない。ライセンスの制限は ROP 側に掛かっているように見える（確かめていないので断定しない）。
- ・ Houdini 自身の形式は完全に往復する。bgeo.sc と geo は 371点がそのまま戻り、位置のずれ 0.000000000、name も残る。Houdini 内のやり取りなら何も失わない。
- ・ obj と ply は位置だけ戻る。371点が 371点で戻り、ずれも 0.000000000。ただし name は消える。骨の名前は失われるので、リグとしては渡せない。
- ・ stl は点が減る。371点が 365点。6点足りない。stl は三角形の集まりしか持てないので、骨の線 (polyline) が落ちている。形だけを渡す形式。
- ・ ファイルの大きさは形式で33倍ちがう。bgeo.sc が 6,231バイト、stl が 207,899バイト。同じ形なのに33倍。bgeo.sc は圧縮された専用形式、stl は三角形を1枚ずつ書き並べる形式だから。
- ・ 結論として、Apprentice のままでは外へ渡せない。形を見せるだけなら obj、Houdini 同士なら bgeo.sc。リグやアニメーションを他のソフトへ渡すには、ライセンスを上げるしかない。これは工夫では回避

できない。

次にやること

- ・実験060で 031～060 の振り返り点検
- ・USD の .usdnc が何なら読めるのかを確かめる
- ・obj で渡せる範囲（形だけ）で何ができるか

obj で渡せる範囲 — 色と UV は残る。自作のアトリビュートは全部落ちる

2026-09-24

何を作ったか

実験058で、Apprentice から外へ出せるのは bgeo.sc / geo / obj / ply / stl の5つだけだと分かった。このうち他のソフトが読めるのは obj と stl。では obj で何が渡せて、何が落ちるのか。1つずつ持たせて往復させた。

形の種類は、ポリゴン・線・点・開いたカーブがそのまま戻った。位置のずれは 0.0000000000。ただし NURBS だけは別で、312点 / NURBS面1枚が 2,232点 / ポリゴン2,160枚になる。点が 7.2倍、ファイルは 705KB。obj は曲面を持ってないので、書き出すときに刻まれる。

アトリビュートは色と UV だけが残った。Cd と uv は戻すが、自分で付けた myvec / myfloat / myint / mystring と pscale は全部消える。グループも消える。

もうひとつ細かい違いがある。法線（N）は書き出す前はバーテックスに乗っていたのに、読み直すと点に乗っている。値は失われないが、置き場所が変わる。

形の種類は残るか

それぞれ1種類だけを持つジオメトリを作り、obj で書き出して読み直した。

種類	点 元	点 後	面 元	面 後	位置のずれ	中身の変化
面（ポリゴン）	122	122	132	132	0.0000000000	Polygon 132 → Polygon 132
線（1本）	9	9	1	1	0.0000000000	Polygon 1 → Polygon 1
点だけ	50	50	0	0	0.0000000000	（面なし）→（面なし）
NURBS の球	312	2,232	1	2,160	—（数が合わない）	NURBSSurface 1 → Polygon 2,160
開いたカーブ	11	11	1	1	0.0000000000	Polygon 1 → Polygon 1

アトリビュートは残るか（面のポリゴンで見る）

点に Cd / myvec / myfloat / myint / mystring / pscale を、バーテックスに N と uv を持たせて往復させた。

置き場所	書き出す前	読み直した後
点	Cd, P, myfloat, myint, mystring, myvec, pscale	Cd, N, P
バーテックス	N, uv	uv
プリミティブ	（なし）	（なし）
点のグループ	firstten	（なし）
面のグループ	（なし）	（なし）

ファイルの大きさ

同じ設定で書き出したときの重さ。

種類	バイト	点あたり
面（ポリゴン）	40,409	331.2 B
線（1本）	791	87.9 B
点だけ	3,974	79.5 B
NURBS の球	705,418	2,261.0 B
開いたカーブ	1,058	96.2 B

わかったこと

- ・ポリゴン・線・点・開いたカーブは、そのまま戻る。位置のずれは4通りすべてで 0.0000000000。数も種類も変わらない。形を渡すだけなら obj で足りる。
- ・NURBS だけは刻まれる。312点 / NURBS面1枚が 2,232点 / ポリゴン2,160枚に変わる。点が 7.2倍。obj は曲面を持ってないので、書き出すときにポリゴンへ変換される。
- ・その代償はファイルの重さに出る。NURBS の球が 705,418バイト。同じ球をポリゴンで作った 40,409バイトの 17倍。点あたりでも 331.2B に対して 2,261.0B。NURBS のまま渡そうとしない。
- ・残るアトリビュートは色と UV だけ。Cd と uv は戻る。自分で付けた myvec / myfloat / myint / mystring は全部消える。pscale も消える。
- ・グループも消える。点のグループ firstten は跡形もない。obj にはグループの仕組みがあるが、Houdini からは書かれていない。
- ・法線は置き場所が変わる。書き出す前はバーテックスに乗っていた N が、読み直すと点に乗っている。値は失われないが、場所が変わる。実験030・036で「normal SOP はバーテックスに書く」ことに何度も掛かってきたが、obj を経由すると点に移る。
- ・つまり obj で渡せるのは「形・色・UV」まで。骨の名前も、重みも、自作の値も渡らない。実験058で分かった「リグは渡せない」が、ここでも別の角度から確かめられた。
- ・点だけのジオメトリも渡せる。50点が 50点で戻る。面が0でも問題ない。ばらまいた位置を他のソフトへ渡す、という使い方はできる。
- ・測り方について。1つの種類だけを持つジオメトリを5通り作って、1つずつ往復させた。全部を1つに混ぜて出すと、何が原因で何が落ちたのか分からなくなる。分けて測る。

次にやること

- ・実験060で 031~060 の振り返り点検
- ・stl で何が渡せるかを同じやり方で測る
- ・obj に書ける情報を自分で足せるか

031～059 の振り返り点検 — 6件すべて一致。1件は「もっと強い結論」に変わった

2026-09-24

何を作ったか

30件ごとの点検。実験030では 001～029 を点検した。今回は 031～059。

同じ条件で走らせ直しても意味がない。同じ結果が出るだけで、それは「再現した」だけであって「正しい」とは言えない。だから条件を変えて測る。式として書いたものは、数字を変えても同じ式に乘るはずだ。乗らなければ、それは式ではなく偶然だった。

6件を選び、すべて別の条件で測り直した。結果は
6件すべて一致。訂正が必要な箇所は見つからなかった。

そして1件は、もとの結論より強いことが言えるようになった。実験052で見つけた「体積の減り = $27.1284\% \times (1 - \cos \theta)$ 」の係数 27.1284 は、骨の長さを 1.0 から 1.5 へ、皮の半径を 0.32 から 0.5 へ変えても、小数点以下4桁までまったく同じだった。しかも角度6通りでの係数のばらつきは 0.0000。つまりこの係数は形の大きさによらない定数で、混ぜ方そのものの性質である。

点検のまとめ

もとの実験と違う条件で測り直したもの。

実験	確かめたこと	もとの記録	測り直し	結果
042	球 (40×40) の表面積	12.53039	12.530387	一致
042	density は面積1あたりの本数	99.678 (半径1)	100.556 / 99.678 / 100.755	一致
045	width は直径 (比 ≈ 1.0)	0.9969～1.0019	0.9999	一致
052	体積の減り = 係数 $\times (1 - \cos \theta)$	係数 27.1284、幅 0.0002	係数 27.1284、幅 0.0000	一致
056	届く距離 = 骨の長さの合計	骨の本数 (長さ1)	差の最大 0.000000000	一致
057	ノード数=点数、配線数=プリミティブ数	一直線の足し算で確認	枝分かれでも 5点 / 6プリミティブ	一致

density は土台の大きさによらないか (実験042)

球の半径を3通りに変え、density は 100 のまま固定した。面積は面から三角形を足して自分で出している。

半径	面積	density	出た本数	本数 ÷ 面積
0.5	3.13260	100	315	100.556
1.0	12.53039	100	1,249	99.678
2.0	50.12155	100	5,050	100.755

体積の減りの係数は、形を変えても同じか (実験052)

骨の長さを 1.0 → 1.5、皮の半径を 0.32 → 0.5、皮の高さを 2.0 → 3.0 に変えた。

角度	体積	減った分	$\div (1 - \cos \theta)$
15°	2.307839	0.9244%	27.1284
30°	2.244710	3.6345%	27.1284
45°	2.144285	7.9457%	27.1284
60°	2.013410	13.5642%	27.1284
90°	1.697449	27.1284%	27.1284
120°	1.381488	40.6926%	27.1284

IK の届く距離は、骨の長さを変えても合計になるか（実験056）

骨1本の長さを 1.0 から 0.7 に変えた。目標は届く距離の3倍の位置に置いている。

関節	骨	合計長	目標まで	先端までの距離	合計長との差
3	2	1.40	4.20	1.400000000	0.000000000
5	4	2.80	8.40	2.800000000	0.000000000
9	8	5.60	16.80	5.600000000	0.000000000

わかったこと

- ・6件すべて一致した。訂正が必要な箇所は見つからなかった。実験030（001～029の点検）では角度の測り方の食い違いが1件見つかったが、今回は無い。

- ・そして1件は、もとの結論より強いことが言えるようになった。実験052の係数 27.1284 は、骨の長さを 1.5倍、皮の半径を 1.56倍、皮の高さを 1.5倍にしても小数点以下4桁までまったく同じ。しかも角度6通りでのばらつきが 0.0000（もとの実験では 0.0002）。

- ・つまり 27.1284% は形の大きさによらない定数だった。「この筒ではこうなる」ではなく「Linear で混ぜるとこうなる」という、やり方そのものの性質。もとの記事では「この条件で 27.1284%」としか言えなかったが、いまは大きさによらないと言える。

- ・density は土台を変えても「面積あたりの本数」だった。半径 0.5 / 1.0 / 2.0 で、本数÷面積が 100.556 / 99.678 / 100.755。面積が16倍になっても比は変わらない。幅 1.077 はばらまきの乱数によるもの。

- ・球の面積は、別のやり方でも 12.530387。もとの記録 12.53039 と小数点以下5桁まで一致。真球 12.566371 の 99.71% で、ポリゴンで近似したぶん内側に寄っている（実験012の結論とも合う）。

- ・width が直径であることも、別の条件で確かめられた。解像度を 600 から 480 へ、太さを 0.03 へ変えて、比は 0.9999。

- ・IK の届く距離は、骨の長さを 0.7 にしても合計長ちょうど。3通りすべてで差が 0.000000000。「骨の本数」ではなく「長さの合計」が正しい言い方だと、ここではっきりした（実験056は長さ1だったので、どちらとも読めた）。

- ・APEX のノード数=点数は、枝分かれでも成り立つ。もとの実験は一直線の足し算だけだった。分岐と合流のある形（ノード5個・配線6本）でも 5点 / 6プリミティブ。

- ・点検の意味について。同じ条件で走らせ直しても「再現した」ことしか分からない。条件を変えて同じ式に乗るかを見ると、それが式なのか偶然なのかが分かる。今回、052 の係数が「形によらない定数」だと分かったのは、条件を変えたから。

次にやること

- ・ 061 以降。新しい分野へ進む
- ・ 次の振り返り点検は実験090で 061～090 を対象に
- ・ 手順ページを増やす

測る処理が、計算より重くなっていた — 1粒ずつ読むのをやめると 740倍速い

2026-09-24

何を作ったか

実験076で、同じ 15,591粒の MPM が、片方では 4.05秒、もう片方では 20秒かかった。違いは「毎フレーム、全部の粒を Python で読んでいたかどうか」だけだった。測るための処理が、測りたい計算より重くなっていた。

読み方を5通り用意して、実験070の動く板（40フレーム）で比べた。読み方ごとに毎回組み直し、計算と読む時間を別々に測った。

15,591粒で1フレームあたり、読まなければ 0.01ms。これまでの実験のやり方（1粒ずつ position() を読む）は 81.67ms で、計算そのもの（1フレーム約 86ms）とほぼ同じだけかかる。速さ v も1粒ずつ読むと 226.34ms で、計算の2.6倍。

全部を1回で受け取ると一気に軽くなる。pointFloatAttribValues は 0.57ms、文字列で受け取って numpy に渡すと 0.11ms。1粒ずつ読むのと比べて 740倍速い。読んだ値（高さの平均）は、どの読み方でも6桁まで同じだった。

粒が少なければ差は小さい。547粒では 1粒ずつでも1フレーム 2.94ms で、計算の数%にとどまる。粒を増やすほど、1粒ずつ読むやり方だけが重くなっていく。

15,591粒を1フレーム読む時間

「計算と比べて」は、40フレーム読む時間 ÷ 読まないときの計算の時間（3.457秒）。

実験077 — 15,591粒の位置を1フレーム読む時間

MPM の動く板（40フレーム）。計算そのものは 1フレーム約 86ms。

読み方	1フレーム	40フレーム	計算と比べて
1粒ずつ position() と v	226.34 ms	9.05 秒	2.6倍
1粒ずつ position()	81.67 ms	3.27 秒	0.95倍
pointFloatAttribValues	0.57 ms	0.023 秒	0.7%
numpy.frombuffer（文字列で受け取る）	0.11 ms	0.004 秒	0.1%

位置だけを読んだ3通りは、読んだ値（高さの平均 0.205530）が6桁まで同じ。

1粒ずつ読むと、計算と同じか、それ以上の時間がかかる。まとめて受け取れば 1% 未満。

粒の数と読み方（実験070の動く板・40フレーム）

「計算」は geometry() を呼んで計算させた時間、「読む」はそのあと値を取り出した時間。読み方ごとに組み直している。間隔 0.12 の「読まない」の読む時間 0.194秒は、最初の1回だけ numpy を読み込んだ時間（読み方の関数の中で import している）。

間隔	粒	読み方	計算の秒	読む秒	1フレームで読むms
0.12	547	読まない	2.226	0.194	4.86
0.12	547	1粒ずつ位置	1.571	0.118	2.94
0.12	547	1粒ずつ位置と v	1.575	0.326	8.15
0.12	547	まとめて	1.641	0.002	0.06
0.12	547	numpy	1.814	0.003	0.07
0.06	4580	読まない	2.813	0.000	0.01
0.06	4580	1粒ずつ位置	2.929	0.971	24.27
0.06	4580	1粒ずつ位置と v	3.316	2.726	68.15
0.06	4580	まとめて	2.835	0.007	0.19
0.06	4580	numpy	2.662	0.003	0.08
0.04	15591	読まない	3.457	0.000	0.01
0.04	15591	1粒ずつ位置	3.975	3.267	81.67
0.04	15591	1粒ずつ位置と v	4.872	9.054	226.34
0.04	15591	まとめて	3.575	0.023	0.57
0.04	15591	numpy	3.237	0.004	0.11

わかったこと

- ・1粒ずつ読むと、計算と同じだけの時間がかかる。15,591粒で1フレーム 81.67ms。計算そのもの（約 86ms）とほぼ同じ。速さ v も1粒ずつ読むと 226.34ms で、計算の2.6倍。
- ・まとめて受け取れば、ほぼ0になる。pointFloatAttribValues で 0.57ms、pointFloatAttribValuesAsString を numpy.frombuffer に渡すと 0.11ms。1粒ずつより 740倍速い。
- ・読んだ値は変わらない。位置だけを読んだ3通り（1粒ずつ・まとめて・numpy）で、最後の高さの平均は 0.338196 / 0.231387 / 0.205530（間隔 0.12 / 0.06 / 0.04）と、6桁まで同じだった。
- ・粒が少なければ気にしなくてよい。547粒では1粒ずつでも1フレーム 2.94ms。粒が 8.4倍（4,580）で 24.27ms、28.5倍（15,591）で 81.67ms と、粒の数にほぼ比例して重くなる。
- ・読む処理が重いと、計算の時間も少し伸びた。15,591粒で、読まないとき 3.457秒、1粒ずつ位置と v を読んだとき 4.872秒。なぜかは確かめていない（メモリの出し入れなどが考えられるが、推測）。
- ・実験076の「滑る塊は20秒かかった」の大半は、この読む時間だった。同じ粒の数の動く板は、最初と最後しか読まないの で 4.05秒だった。計算の重さを比べるときは、読むのは必要なフレームだけにする。
- ・計算の時間には、1回ごとに ±0.3～0.6秒ほどの揺れがある。間隔 0.12 で 1.57～2.23秒。0.1秒を争う比べ方をするなら、何度か測って中央値を取る。
- ・実用の結論。粒やポイントの値を Python で集計するときは、pointFloatAttribValues（できれば numpy.frombuffer）でまとめて受け取る。これまでの実験の測り方も、粒が多い場面ではこちらに切り替える。

次にやること

- ・hou_tools に「まとめて読む」関数を入れて、これまでの測り方を置き換える

- ・ ボリュームの値 (allVoxels / voxelSlice) でも同じ差が出るか
- ・ Liquid の暴れ始め (実験073) は、粒の細かさで動くか

道に沿って物を並べる — N

だけでは坂で杭が倒れる。up を足せば 0度。Pack で 250倍速い

2026-09-24

何を作ったか

S字の道（曲線）に沿って、両側に杭を並べた。手順ページに書く前に、間隔・向き・重さの3つを測った。

間隔。resample の Length に 1.0 を入れても、点の間隔はちょうど 1.0 にはならなかった。長さ 25.02 の道が27点・26区間に切られ、間隔は約 0.96 ($25.02 \div 26 = 0.962$)。Length は「これより長くしない」という上限で、全体をそれ以下の長さで等分する。最後の区間だけ短くなることもなく、Maintain Last Vertex の入り・切りでも変わらなかった。

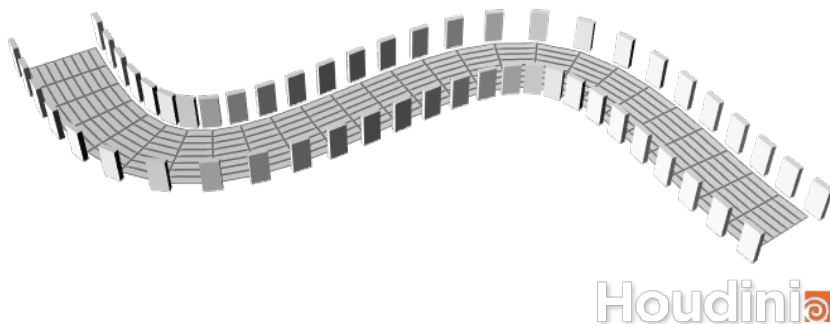
向き。resample で道の向き (tangentu) を点に付け、それを N に入れて copytopoints で配ると、杭の前 (+Z) は道の向きとずれ 0.0000度で一致した。N を入れなければ、杭はすべて同じ向きのままで、道とのずれは平均 84度。

平らな道では、N だけでも杭は傾かなかった。ところが道に起伏（高さ ± 1.2 ）を付けると、N だけでは杭が横に傾き、最大 49度・平均 19度。N と一緒に up に (0, 1, 0) を入れると、傾きは 0.0000度になった。

重さ。細かい球 (48×48) を配ると、Pack and Instance の差が大きい。1万個で、切りだと 2,210万点・1,878MB・714ms、入りだと 1万点・4MB・2.9ms。約250倍速く、メモリは約460分の1。

完成した道

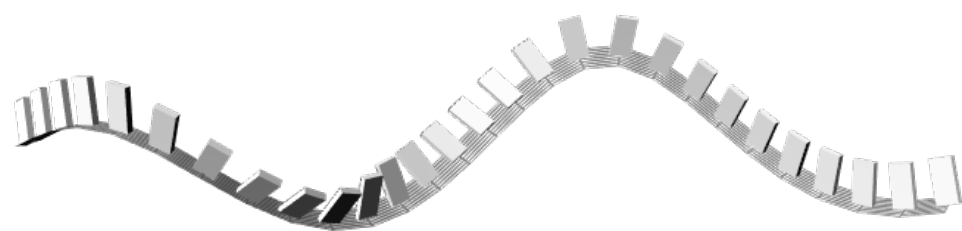
長さ 20 の直線を $z = 3 \sin(0.35 x)$ で曲げ、resample (Length 1.0) で切った。各点を道の横向き (N $\times$ up) $\times \pm 1.2$ ずらして2本にし、 $0.15 \times 1.0 \times 0.5$ の杭を配った。道の面は sweep の Ribbon (幅 2)。



54本の杭が、道の両側で道の向きにそろって立っている。

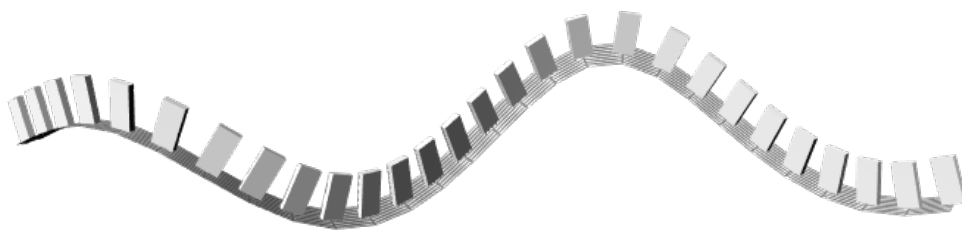
B. 起伏のある道での杭の向き

上が N だけ、下が N と up。道の高さは ±1.2 で上下する。2枚とも同じカメラ。



Houdini

N だけ。谷のあたりの杭が横に倒れている（最大 49度）。



Houdini

N と up。坂に合わせて前後には傾くが、横には倒れない（0.0000度）。

道	向きの入れ方	前と道の向きのずれ（最大）	横の傾き（最大）	横の傾き（平均）
平ら	N と up	0.0000度	0.0000度	0.0000度
平ら	N だけ	0.0000度	0.0000度	0.0000度
平ら	入れない	135.95度	0.0000度	0.0000度
起伏あり	N と up	0.0000度	0.0000度	0.0000度
起伏あり	N だけ	0.0000度	48.96度	19.02度
起伏あり	入れない	134.83度	0.0000度	0.0000度

A. resample の Length 1.0 で切った間隔

間隔は点どうしの直線距離。道は曲がっているの、道に沿った長さ（0.962）より少し短い。

Maintain Last Vertex	道の長さ	点の数	間隔（最後以外）	最後の間隔
切り	25.0217	27	0.9576～0.9624	0.9621

Maintain Last Vertex	道の長さ	点の数	間隔（最後以外）	最後の間隔
入り	25.0217	27	0.9576~0.9624	0.9621

C. 細かい球（48 × 48・2,210点）を配る

毎回まっさらに組み直し、配る直前までを計算させてから、copytopointsの最初の計算を3回測った中央値。

配る数	Pack and Instance	計算の時間	点の数	メモリ
1,024	切り	75.4 ms	2,263,040	192.3 MB
1,024	入り	0.3 ms	1,024	0.4 MB
10,000	切り	713.8 ms	22,100,000	1,878.0 MB
10,000	入り	2.9 ms	10,000	4.1 MB

わかったこと

- ・ resample の Length は「上限」で、全体を等分する。1.0 を入れると、長さ 25.02 の道が26等分されて間隔は約 0.96。ちょうど 1.0 にしたいなら、道の長さのほうを合わせる。最後の区間だけ短くなることはなかった。
- ・ N に道の向きを入れれば、前はぴったり道を向く。resample の Create Tangent Point Attribute で tangentu を作り、@N = normalize(v@tangentu)。ずれは 0.0000度。copytopoints は形の +Z を N に合わせる（実験008）。
- ・ 坂のある道では、N だけでは杭が横に倒れる。高さ ±1.2 の起伏で、横の傾きが最大 49度・平均 19度。up に (0, 1, 0) を足すと 0.0000度。平らな道では N だけでも 0度だったので、平らなところで確かめただけでは気づけない。
- ・ 道の横向きは N × up で作れる。cross(@N, v@up) を正規化して ±1.2 ずらすと、道の両側に並ぶ。
- ・ 細かい形をたくさん配るなら、Pack and Instance を入れる。1万個で 714ms → 2.9ms（約250倍）、1,878MB → 4.1MB（約460分の1）。点の数は 2,210万 → 1万。
- ・ 測り方で2回つまずいた。1つ目。配る先の板をわずかに動かして計算し直させると、Pack 切りでも 9.5ms と出た。位置だけが変わったときは、全部を作り直してないらしい（確かめてはいない）。毎回組み直して測ると 714ms だった。2つ目。点の数を len(g.points()) で数えると、2,210万点を Python の一覧にする時間（約8.8秒）が乗った。intrinsicValue("pointcount") で数えれば乗らない（実験077と同じ話）。
- ・ 計算の時間は、道・杭54本・道の面208枚で 0.097秒。

次にやること

- ・ 配る物に大きさや向きのばらつきを付ける（pscale・orient）
- ・ 曲線の曲がり具合に合わせて、間隔を変える
- ・ 振り返り点検（061~090）を 090 で行う

061～090 の振り返り点検 — 7件を測り直して全部一致。訂正は2件、黙って無視される設定は8件に

2026-09-24

何を作ったか

30件ごとの点検の3回目（実験030・060に続く）。やることは4つ。書いた数字を測り直す、あとで覆った結論を数える、「黙って無視される設定」の一覧を更新する、直したほうがよい点を洗い出す。

測り直しは7件を選んだ。重い測定ではなく、数字がはっきり決まっているものを選んでいく。結果は7件すべて記録と同じ値だった。滑った距離 3.260808、運ばれた距離 5.336132 は小数6桁まで、ドミノが倒れきる 1.500秒、液体が暴れ始める28フレーム目、道を切った点 27個も同じ。回る台の境目（ω 1.25 は乗ったまま、1.30 は滑り出す）も同じ向きに出た。

この期間に覆った結論は2件。実験068の「Liquid は摩擦 0.25 では暴れない」は、40フレームで止めていたからで、実験073で60フレームまで回すと46フレーム目から暴れた。実験071の「摩擦は 0.3～0.4 ぶんしか効かない」は、実験075で「粒の間隔 0.12 のとき」という条件が付いた。どちらも元の記事の冒頭に注記を入れてある。

「黙って無視される設定」は8件になった。044のガイド、050の可変長入力、062の materialpreset、066の風速、070のコライダ Type、079の DOP の表示フラグと Emission Type、084の packed に付けた初速。どれもエラーも警告も出ない。見つけ方はいつも同じで、入力を極端に振って出力が動くかを見る。

測り方そのものも2つ直した。実験077で「1粒ずつ読むと計算より重くなる」と分かり、hou_tools に「まとめて読む」関数を入れた。実験082では、点の数を len(g.points()) で数えると 2,210万点で8.8秒かかることが分かり、intrinsicValue("pointcount") に変えた。

サイト側は、手順ページが 15本から 36本に増えた。効率化の項目は 28。Claude 版は1つの版に置けるファイルが 512 までなので、中身が同じ画像を1枚にまとめる仕組みを入れた。

測り直し（7件）

同じスクリプトを別のプロセスから呼び直して測った。実験071だけは数値ではなく「乗ったまま／滑り出す」の向きで見ている。

実験090 — 061～090 を測り直す

7件すべて、記録と同じ値が出た。

確かめたこと	記録	測り直し	結果
067 滑った距離（摩擦0.25）	3.260808	3.260808	一致
070 運ばれた距離（板4・摩擦1）	5.336132	5.336132	一致
071 境目（ω1.25は乗る/1.30は滑る）	乗る / 滑る	乗る / 滑る	一致
073 液体が暴れ始めるフレーム	28	28	一致
079 空気抵抗の式とのずれ（最大）	0.0000	0.0000	一致
084 ドミノが倒れきる秒（12枚・間隔0.3）	1.500	1.500	一致
082 道を切った点の数	27	27	一致

同じスクリプトを別のプロセスから呼び直して測った。

7件すべて、記録と同じ値が出た。

確かめたこと	記録	測り直し	結果
067 滑った距離（摩擦 0.25）	3.260808	3.260808	一致
070 運ばれた距離（板 4・摩擦 1）	5.336132	5.336132	一致
071 境目（ ω 1.25 は乗る / 1.30 は滑る）	乗る / 滑る	乗る / 滑る	一致
073 液体が暴れ始めるフレーム	28	28	一致
079 空気抵抗の式とのずれ（最大）	0.0000	0.0000	一致
084 ドミノが倒れきる秒（12 枚・間隔 0.3）	1.500	1.500	一致
082 道を切った点の数	27	27	一致

この期間に覆った結論

どちらも元の記事の冒頭に注記を入れ、レポート（ポップアップ）にも同じ訂正を書いた。

元の実験	訂正した実験	内容
068	073	「Liquid は摩擦 0.25 では暴れない」は、40フレームで止めていたため。60フレームまで回すと F46 から飛び始める
071	075	「摩擦は 0.3～0.4 ぶんしか効かない」は、粒の間隔 0.12 のときの話。0.08 で 0.65、0.04 で 0.79 に上がる

黙って無視される設定（8件）

どれもエラーも警告も出ない。入力を極端に振って、出力が動くかどうかで見つけた。

実験	設定	入れても起きないこと
044	ガイドのジオメトリ	毛がガイドに従わない
050	可変長の入力	2本目以降が無視される
062	materialpreset	スクリプトから押しても材質が変わらない
066	風速	値を変えても結果が同じ
070	mpmcollider の Type	Static のままだと板を動かしても伝わらない
079	DOP の表示フラグ	立てないと粒が1つも生まれない
079	Emission Type の All Points	Impulse Count を入れても1点1粒しか出ない
084	packed に付けた初速（v）	プリミティブに書いても動かない。点に書く

わかったこと

- ・測り直した7件は、すべて記録と同じ値だった。小数6桁まで同じもの（3.260808 / 5.336132）を含む。別のプロセスから同じスクリプトを呼び直しても、結果は変わらない。
- ・覆った結論は2件で、どちらも「測る範囲が足りなかった」のが原因。068は40フレームで止めていた、071は粒の細かさを1通りしか試していなかった。条件を1つしか試していない結論には、条件を書き添える。
- ・「黙って無視される設定」は8件になった。061～090
だけで4件増えた（070・079が2件・084）。組んで動かないときは、まずこの型を疑う。
- ・測り方の落とし穴も2つ直した。1粒ずつ読むと計算より重くなる（実験077）、点の数を Python 側で数えると8.8秒かかる（実験082）。どちらも `hou_tools` の `point_array()` と `intrinsicValue("pointcount")` に置き換えた。
- ・手順ページは 15本から
36本になった。実験079以降は「手順ページ向き」の題材を選び、実験ごとに手順を1本足している。
- ・残っている宿題。(1) 手順「テクスチャを貼る」だけ、元になるシーン（.hipnc）と Houdini の画面が無い。(2) 実験083～089 で足した手順7本も、Houdini の画面をまだ撮っていない。(3) Claude 版のファイル数が上限 512 に近い（496）。
- ・点検にかかった時間は、測り直し7件で合わせて 12.5秒だった。

次にやること

- ・手順7本の Houdini 画面を撮る
- ・テクスチャの手順にシーンを作る
- ・091～120 の方向を決める（次の点検は120）

面を半分にすると、形のずれはおよそ2倍になる — 25%まで減らしてもずれは幅の 0.1194%

2026-09-24

何を作ったか

polyreduce は「残す割合」を決めると、そこまで面を間引く。軽くなるのは当たり前だが、形はどれだけ元から離れるのか。3540面の形を 80/50/25/10/5/2% まで減らして測った。

ずれの測り方は、減らしたあとの各点から元の面までの最短距離（VEX の xyzdist）。その平均と最大を、元の形の幅で割って割合にした。

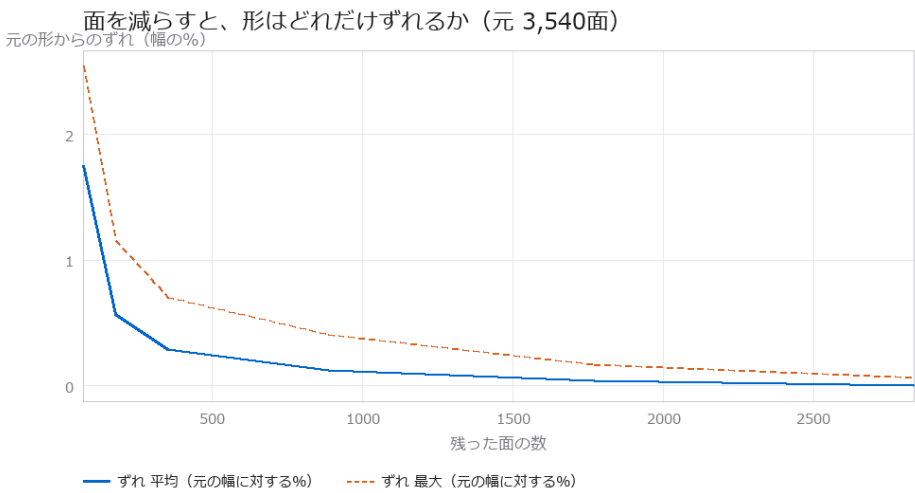
25%まで減らすと、面は 3540 から 885 へ。ずれの平均は幅の 0.1194%、最大でも 0.4108%。見た目にはほとんど分からない大きさに収まっている。

減らしすぎると急に崩れる。2%（70面）では平均 1.7497%、最大 2.5434%。50%から2%までの区間では、面の数が半分になるたびにずれがおよそ2倍になっていた。

時間は、どの割合でもほとんど変わらない（0.0062～0.0138秒）。減らす量を増やしても遅くならない。

残す割合ごとの、面の数と形のずれ

元は球（60×60）に mountain をかけた 3540面・幅 1.995536。ずれは、減らしたあとの各点から元の面までの最短距離。「幅の%」は、その距離を元の形の幅で割ったもの。



25%まではずれが小さいまま。そこから先は、面を半分にするたびにずれがおよそ2倍になる。

残す%	面	点	ずれ 平均	幅の%	ずれ 最大	幅の%	秒
80	2832	2384	0.000087	0.0044%	0.001369	0.0686%	0.006
50	1769	1110	0.000853	0.0428%	0.003420	0.1714%	0.010
25	885	481	0.002383	0.1194%	0.008198	0.4108%	0.012
10	353	182	0.005794	0.2903%	0.014111	0.7071%	0.014
5	177	92	0.011303	0.5664%	0.023076	1.1564%	0.013
2	70	37	0.034915	1.7497%	0.050754	2.5434%	0.014

わかったこと

- ・ 25%まではほとんど変わらない。面を 3540 から 885 に減らしても、ずれの平均は幅の 0.1194%。半分（1769面）なら 0.0428%。
- ・ 面を半分にすると、ずれはおよそ2倍。1769→885面で 2.79倍、885→353面で 2.43倍、353→177面で 1.95倍、177→70面で 3.09倍。
- ・ 減らす量を増やしても遅くならない。0.0062秒から 0.0138秒の間で、残す割合との関係は見えなかった。

次にやること

- ・ ずれの2倍という関係が、もっと角のある形でも成り立つか
- ・ polyreduce と remesh で、同じ面数にしたときどちらがずれないか

辺の長さを半分にすると、面は約4倍・ずれは約1/4・時間は約4倍

2026-09-24

何を作ったか

remesh は「目指す辺の長さ」を決めて、そこに近い三角形で張り直す。面積あたりの三角形の数で考えれば、辺を半分にすると面は4倍になるはず。球（40×40、1560面）を、辺 0.2 / 0.1 / 0.05 / 0.025 で張り直して測った。

面の数は 678 → 2908 → 11518 → 47238。前の段からの倍率は 4.289 / 3.961 / 4.101 で、いずれも4倍の前後だった。

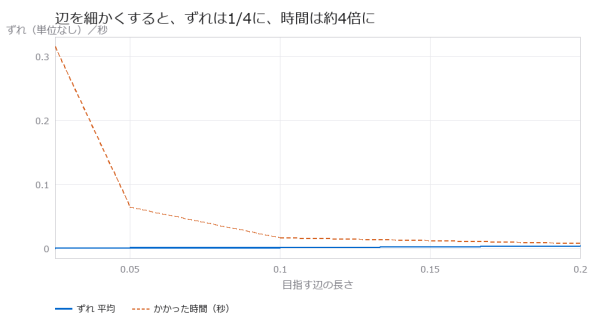
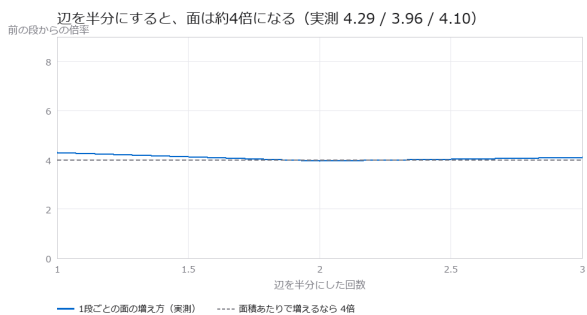
元の形からのずれは 0.003694 から 0.000058 まで小さくなる。1段ごとの縮み方は 4.49倍 / 3.59倍 / 3.95倍。こちらも約4分の1ずつ。

時間は 0.0078秒 から

0.3146秒。面の数に比例して増えている。細かくすることの代金は、面の数そのままに乘る。

目指す辺の長さごとの、面の数・ずれ・時間

元は球（40×40）の 1560面。「面の倍率」は前の段からの比。ずれは各点から元の面までの最短距離。



1段ごとの面の増え方は4倍の前後に収まる。空間ではなく面積あたりで増えていることになる。

ずれは約1/4ずつ小さくなり、時間は約4倍ずつ増える。

目指す辺	面	点	面の倍率	ずれ 平均	ずれ 最大	秒
0.2	678	341	—	0.003694	0.008615	0.008
0.1	2908	1456	4.289	0.000823	0.003000	0.017
0.05	11518	5761	3.961	0.000229	0.001321	0.065
0.025	47238	23621	4.101	0.000058	0.000660	0.315

わかったこと

- ・面は4倍ずつ増える。実測 4.289 / 3.961 / 4.101。辺の長さを半分にすると、面積あたりの三角形の数が4倍になるという見方と合う。
- ・ずれは約1/4ずつ小さくなる。4.49倍 / 3.59倍 / 3.95倍。面を4倍にして、ずれが1/4。増やした分がそのまま精度になっている。
- ・時間も約4倍ずつ。0.0078秒 → 0.3146秒。面の数に比例するので、細かさを上げるほど代金は素直に増える。

次にやること

- ・ 同じ面数にしたとき、remesh と polyreduce でどちらがずれないか
- ・ 角のある形（箱や文字）では、辺の長さを決めるやり方が通じるか

Force Total Count はぴったり合う — 10通り試して差は1つも出なかった

2026-09-24

何を作ったか

scatter の「Force Total Count」は、指定した数をそのまま出すという意味のつまみ。面の大きさが場所によって違う形でも、本当にぴったりになるのか。100点から100万点まで、2つの形で測った。

結果は、**10通りすべてで差が +0**。平らな板（1面）でも、凹凸のある球（3,540面）でも、指定した数がそのまま出た。種を変えても数は変わらない（1万点で3回、いずれも 10,000点）。

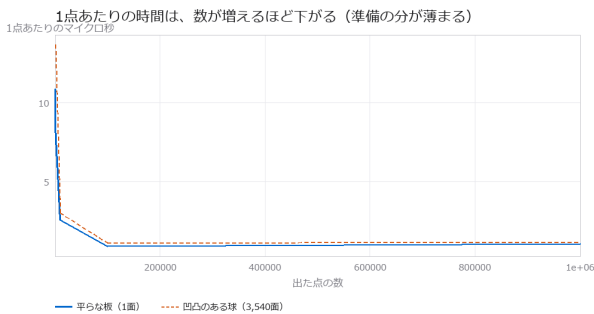
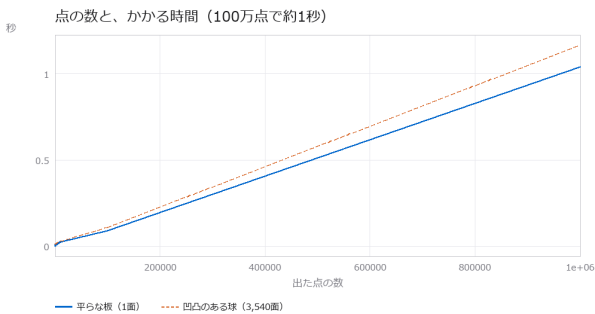
時間は点の数にほぼ比例する。板では 1万点で 0.0254秒、100万点で 1.0384秒。100倍の点で 40.9倍。

1点あたりで見ると、数が少ないうちは割高になる。板では100点で 10.873マイクロ秒、100万点で 1.038マイクロ秒。準備にかかる固定の分が、点の数で薄まっていく。

面の数が多いほうがわずかに遅い。1万点で板が 0.0254秒、球が 0.03秒。ただし差は小さく、点の数のほうがずっと効く。

指定した数と、実際に出た数・時間

Force Total Count を入れ、Total Count に左の数を指定した。球の100点だけ 0.3371秒と外れて見えるが、これは上流の mountain を初めて計算した分が乗っているため。この行だけは時間の比べ物にならない。



時間は点の数にほぼ比例する。100万点で約1秒。

1点あたりの時間は、数が増えるほど下がっていく。

形	指定	出た数	差	秒	1点あたりμ秒
平らな板（1面）	100	100	+0	0.001	10.873
平らな板（1面）	1,000	1,000	+0	0.008	8.425
平らな板（1面）	10,000	10,000	+0	0.025	2.537
平らな板（1面）	100,000	100,000	+0	0.090	0.898
平らな板（1面）	1,000,000	1,000,000	+0	1.038	1.038
凹凸のある球（3, 540面）	100	100	+0	0.337	3371.241
凹凸のある球（3, 540面）	1,000	1,000	+0	0.014	13.664
凹凸のある球（3, 540面）	10,000	10,000	+0	0.030	3.004

形	指定	出た数	差	秒	1点あたりμ秒
凹凸のある球（3, 540面）	100,000	100,000	+0	0.111	1.11
凹凸のある球（3, 540面）	1,000,000	1,000,000	+0	1.166	1.166

種を変えても数は変わらない（1万点）

Seed だけを変えて3回。位置は変わるが数は同じ。

Seed	出た数	秒
1	10,000	0.029
2	10,000	0.029
3	10,000	0.030

わかったこと

- ・数はぴったり合う。10通り試して、差はすべて+0。面の大きさがばらついた形でも、指定した数がそのまま出た。
- ・時間は点の数に比例する。板では1万点 0.0254秒 → 100万点 1.0384秒。100倍の点で 40.9倍。
- ・少ない点ほど割高。1点あたりは板の100点で 10.873マイクロ秒、100万点で 1.038マイクロ秒。準備の固定分が薄まっていく。少しだけ散らすのを何度も繰り返すより、一度にまとめて散らすほうが得。

次にやること

- ・Density Scale やアトリビュートで濃さを変えても、合計はぴったり合うか
- ・Force Total Count を切ったときは、どれだけずれるか

pack を入れると、点は362分の1・ファイルは198分の1になる — 1万個で 22.5MB が 0.114MB

2026-09-24

何を作ったか

copytopoints の「Pack and Instance」は軽くなると言われる。どこがどれだけ軽くなるのかを、3つの数で測った。持っている点と面の数、計算にかかる時間、ディスクに書いたときのファイルの大きさ。

複製する形は球（362点・380面）。板の上に 100 / 1,000 / 10,000 個並べた。

1万個のとき、pack を切ると 3,620,000点・3,800,000面。入れると

10,000点・10,000面で、**点の数は

362分の1**。複製1つが1点になるので、並べた数がそのまま点の数になる。

計算は 0.1289秒 から 0.0017秒へ。**76倍速い。** 書き出しは 0.5272秒 から 0.0156秒。

ファイルは 22.537MB から 0.114MB。**198分の1。**

中身を1つだけ持ち、あとは置き場所だけを並べているので、数を増やしても増え方がゆるい。

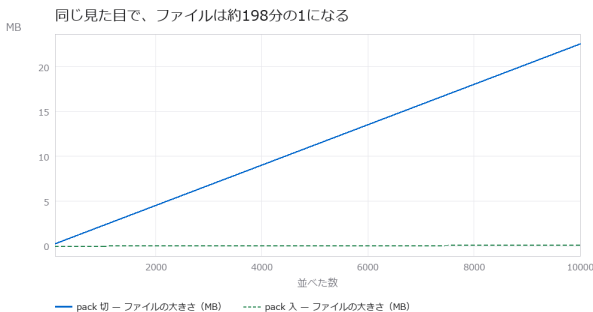
代わりに、中身を直接触れなくなる。点や面を選んで動かしたいときは unpack

で開く必要があり、そこで軽さは失われる。

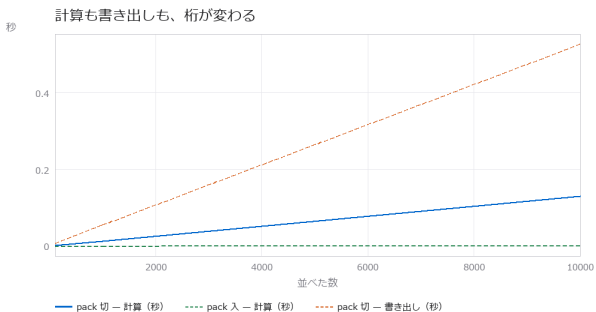
pack の入／切で、点・面・時間・ファイルの大きさ

複製する形は球（20×20、362点・380面）。並べ先は 10×10 の板に scatter

で散らした点。ファイルの大きさは .bgeo.sc に書いて測り、そのあと消した。



ファイルの大きさは桁が変わる。1万個で 22.5MB が 0.114MB。



計算も書き出しも、pack を入れるとほぼ平らなまま。

並べた数	pack	点	面	計算 秒	書き出し 秒	MB
100	切	36,200	38,000	0.002	0.007	0.226
100	入	100	100	0.000	0.001	0.008
1,000	切	362,000	380,000	0.012	0.055	2.253
1,000	入	1,000	1,000	0.000	0.002	0.019
10,000	切	3,620,000	3,800,000	0.129	0.527	22.537
10,000	入	10,000	10,000	0.002	0.016	0.114

わかったこと

- ・点の数は並べた数と同じになる。1万個で 3,620,000点 → 10,000点、362分の1。複製1つが1点として扱われる。
- ・ファイルは198分の1。22.537MB → 0.114MB。中身を1つだけ持ち、残りは置き場所だけを並べている。
- ・計算は76倍速い。0.1289秒 → 0.0017秒。書き出しも 0.5272秒 → 0.0156秒。
- ・代わりに中身を触れなくなる。点や面を選んで動かすには unpack で開く必要があり、そこで軽さは失われる。並べたあと形を変えないなら、入れておいて損はない。

次にやること

- ・ unpack で開いたとき、元の重さに戻るのか（それ以上になるのか）
- ・ pack したまま、まとめて色や大きさを変えられる範囲はどこまでか

091～120 の振り返り点検 — 30本すべて流し直し、結果の数字は1つを除いて全部一致

2026-09-24

何を作ったか

30件ごとの点検（030・060・090 に続いて4回目）。090 までは数件を選んで測り直したが、091 からの台本はどれも数秒で終わるので、**30本すべてを流し直した**。記録（out/NNN_*.json）を控えに退避し、台本を hython

で流し、新しい値と控えを数字1つずつ比べて、最後に控えを戻した（記録は変えていない）。

比べた値は 27,217 個（うち 24,607 個は実験116 の UV の座標）。かかった時間は全部で 61 秒。

違ったのは時間の項目 25 個と、それ以外の 2 個だけ。

- 実験097: パックして書き出したファイルの大きさが 119,599 → 119,600 バイト（1バイト差）。同じものを書いても、ファイルの大きさはぴったり同じにならないことがある（記事の結論「パックすると小さくなる」は変わらない） - 実験101: 記録に、台本が書かない項目（side =

2.0）が1つ混ざっていた。あとから手で足したもの。値は既定と同じで、記事の数字は変わらない

時間の項目（秒・1点あたりのマイクロ秒）は、流すたびに揺れるので比べていない。実験096 の最も遅い行は 3371 → 1418 マイクロ秒と大きく動いた。時間の値は1回の測定では目安にとどまる。

あとの実験で結論に条件が付いたもの: 1件。

実験115（到着時間が縦横の道のりになる）の理由を、実験119 で確かめた（distancealonggeometry の Edge と同じ値）。記事115 に注記を足した。

解説の訂正: 1件。 copyxform の説明を「同じ変形を重ねる」と書いていたが、実験110 で「値を i 倍して1回かける」と分かったので直した。

検算で見つけた落とし穴: 091～120 で8件（boolean は poly 以外だと何もしない 098、carve の First U は入切 104、copyxform は輪にならない 110、timeblend の v は1秒あたり 113、attribfill の到着時間は辺の道のり 115、revolve は裏返しになる 118、box の Use Divisions は線の籠 120、素性の控えがメニューを12個で切っていた 112）。

実験ごとの測り直し

時間の項目（sec・seconds・us_per_point など）は除いて比べた。許す差は 0.000001（相対）。

091~120 を全部流し直して、記録と比べた（数字1つずつ）

091 一致 44 値	092 一致 25 値	093 一致 59 値	094 一致 40 値	095 一致 65 値	096 一致 56 値
097 要確認 44 値・1件	098 一致 92 値	099 一致 68 値	100 一致 176 値	101 要確認 59 値・1件	102 一致 63 値
103 一致 112 値	104 一致 118 値	105 一致 140 値	106 一致 83 値	107 一致 243 値	108 一致 76 値
109 一致 117 値	110 一致 113 値	111 一致 76 値	112 一致 81 値	113 一致 57 値	114 一致 129 値
115 一致 108 値	116 一致 24,607 値	117 一致 144 値	118 一致 67 値	119 一致 105 値	120 一致 50 値

30本のうち28本は、時間以外の数字がすべて一致。

実験	台本	比べた値	時間以外のずれ	秒
091	091_polyreduce.py	44	0	1.2
092	092_remesh.py	25	0	1.4
093	093_smooth_volume.py	59	0	1.7
094	094_vdb_voxel.py	40	0	1.3
095	095_fuse.py	65	0	1.1
096	096_scatter.py	56	0	3.7
097	097_pack.py	44	1	1.7
098	098_hole.py	92	0	5.4
099	099_subdivide.py	68	0	1.3
100	100_peak.py	176	0	2.3
101	101_area.py	59	1	2.2
102	102_extrude.py	63	0	1.0
103	103_rotate_order.py	112	0	1.0
104	104_resample.py	118	0	1.0
105	105_spiral.py	140	0	1.2
106	106_randomize.py	83	0	1.6
107	107_centroid.py	243	0	1.1
108	108_triangulate.py	76	0	6.8
109	109_grouperexpand.py	117	0	1.1
110	110_copyxform.py	113	0	1.0
111	111_jitter.py	76	0	1.5
112	112_vdb_csg.py	81	0	6.6
113	113_timeblend.py	57	0	1.0

実験	台本	比べた値	時間以外のずれ	秒
114	114_sweep_volume.py	129	0	1.9
115	115_attribfill.py	108	0	1.2
116	116_uvflatten.py	24,607	0	2.4
117	117_relax.py	144	0	1.5
118	118_revolve.py	67	0	1.2
119	119_surface_distance.py	105	0	1.4
120	120_thickness.py	50	0	3.4

ずれた値（時間以外）

実験	ファイル	項目	記録	流し直し
097	097_stats.json	.rows[5].bytes	119599	119600
101	101_stats.json	.side	2.0	None

わかったこと

- ・ 30本すべて流し直せた。全部で約1分。091 以降の台本は、1本数秒で結果を出す作りになっている。
- ・ 結果の数字は、ファイルの大きさ1バイトを除いて全部一致。
- ・ 時間の値は揺れる。最も遅い行では2倍以上動いた。速さの比べは、同じ回で並べて測る。
- ・ 記録を手で直さない。台本が書かない項目が1つ混ざっていた。直すなら台本を直して流し直す。
- ・ 次の点検は150。

次にやること

- ・ 時間の項目を、3回流した中央値で記録するようにする
- ・ 点検の台本を、次回（150）もそのまま使えるように番号の範囲を引数にする

最初から式が入っているつまみは SOP 全体で488個 — set しても式が残り、値は変わらない

2026-09-24

何を作ったか

実験133 で、ray の Direction に最初から式が入っていて、Python の set が効かないと分かった。同じ落とし穴がどれだけあるかを、SOP の全ノード 1,031 種類を1つずつ作って数えた（8.5 秒）。

****最初から式が入っているつまみは 488 個、176 種類のノードにあった。****
式の中身は、時間（\$F・\$T など） 275 個、ほかのつまみの参照（ch） 135 個、その他 67 個、属性（@N など） 11 個。多いのは \$FSTART・\$FEND（フレーム範囲）で、キャッシュや読み込みの箱に集まっている。

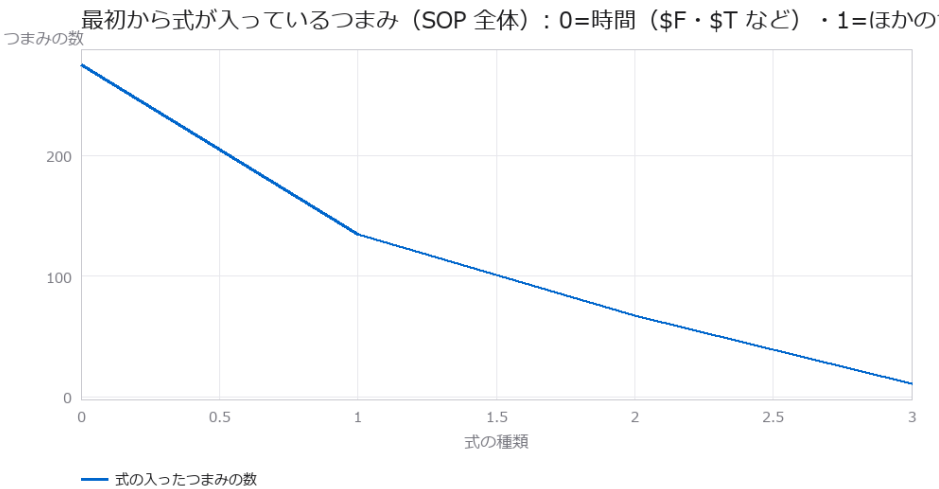
このサイトで解説しているノードでは 69 個。属性を読む式（@N など）は SOP 全体で 11 個で、ray の Direction がその代表。

****式の入ったつまみに set すると、式は残り、値は変わらない。**** ray の dirty で、set(-1) の前は 式 @N.y・値 0、後も 式 @N.y・値 0。deleteAllKeyframes() で式を消してから set すると、値 -1 になった。

スクリプトでつまみを入れるときは、keyframes() で式の有無を確かめ、あれば消してから入れる。

このサイトで解説しているノードの、式の入ったつまみ

式は最初の80文字まで。



式の種類ごとの数。時間の式が大半。

ノード	つまみ	画面の名前	式
alembic	frame	Frame	\$FF
alembic	fps	Frames per Second	\$FPS

ノード	つまみ	画面の名前	式
attribremap	descriptiveparm	Label	chs("inname") + ifs(strcmp(chs("outname"), ""), " -> " + chs("outname"), "")
cache	range1	Start/End/Inc	\$FSTART
cache	range2	Start/End/Inc	\$FEND
cache	index	Index	\$F
delete	rangeend	Start / End	\$N
dopimport	integrateovertime	Integrate Over Time	1/\$FPS
dopnet	timestep	Timestep	ch("timescale")/(ch("subst ep")*\$FPS)
dopnet	timeoffset	Offset Time	(ch("startframe")-1)/\$FPS
file	index	Sequence Index	\$FF-ch("f1")
filecache	f1	Start/End/Inc	\$FSTART
filecache	f2	Start/End/Inc	\$FEND
filecache	clampfirst	Clamp First Frame	ch("f1")
filecache	clamplast	Clamp Last Frame	ch("f2")
filecache	index	Sequence Index	\$FF-1
filecache	frame	Frame	{ if(ch("timedependent")) { rawframe = \$T * \$FPS + 1;
filecache	descriptivelabel	Descriptive Label	ifs(ch("filemethod"),chsraw ("file"),strreplace(strreplac e(chs("cachename"),chs("
filecache	sopoutput	Output File	{ if(ch('filemethod')) { return chs('file'); } else
mpmsurface	sdferode	Erode	ch("sdfdilate")
particlefluidsurface	doerode	Label	ch("dodilate")
particlefluidsurface	erodeoffset	Erode	ch("dilateoffset")
particlefluidsurface	erodemask	Mask	ch("dilatemask")
pointvelocity	framerangemin	Frame Range	\$FSTART
pointvelocity	framerangemax	Frame Range	\$FEND
polywire	segyscale1	Segment Scales	1.0 / \$NSEG
polywire	segyscale2	Segment Scales	1.0 - 1.0 / \$NSEG
primitive	px	Pivot Translate	\$CEX
primitive	py	Pivot Translate	\$CEY
primitive	pz	Pivot Translate	\$CEZ
primitive	upvectorx	Up-Vector	@N.x
primitive	upvectory	Up-Vector	@N.y
primitive	upvectorz	Up-Vector	@N.z

ノード	つまみ	画面の名前	式
primitive	diffrr	Color	@Cd.r
primitive	diffg	Color	@Cd.g
primitive	diffb	Color	@Cd.b
primitive	alpha	Alpha	@Alpha
primitive	crease	Crease	@creaseweight
primitive	metaweight	Weight	\$WEIGHT
primitive	prblur	Particle Blur	1/\$FPS
ray	dirx	Direction	@N.x
ray	diry	Direction	@N.y
ray	dirz	Direction	@N.z
rdbbulletsolver	guidecaptureframe	Guide Capture Frame	ch("startframe")
rdbbulletsolver	emission_cachemaxsize	Emission Cache Memory (MB)	ch("cachemaxsize")
rdbbulletsolver	collision_solvefirstframe	Solve on Creation Frame	ch("solvefirstframe")
rdbbulletsolver	guide_startframe	Start Frame	ch("startframe")
rbdconfigure	bboxonoff	Show Bbox	ch("boundingtype")==0
rbdconfigure	shrinkamount1	Shrink Amount	ch("collisionmargin1")
rbdconfigure	density1	Density (kg/m3)	chs("material"+opdigits(\$CH))
rbdmaterialfracture	enabledetail_	Enable Detail	ch("concrete_edgedetail")
rbdmaterialfracture	enabledetail__2	Enable Detail	ch("concrete_edgedetail")
rbdmaterialfracture	enabledetail_2	Enable Detail	ch("concrete_edgedetail")
rbdmaterialfracture	enabledetail1	Enable Detail	ch("concrete_edgedetail")
rbdmaterialfracture	enabledetail2	Enable Detail	ch("concrete_edgedetail")
retime	frame	Frame	\$FF
retime	time	Time	\$T
retime	startframe	Start Frame	ch("inputrangex")
retime	outputrangex	Output Frame Range	\$FSTART
retime	outputrangey	Output Frame Range	\$FEND
retime	timestep	Input Timestep	1 / \$FPS
timeshift	frame	Frame	\$F
timeshift	time	Time	\$T
timeshift	frange1	Start/End Frame	\$FSTART
timeshift	frange2	Start/End Frame	\$FEND
timeshift	trange1	Start/End Time	\$TSTART
timeshift	trange2	Start/End Time	\$TEND
trace	copframe	COP Frame	\$F

ノード	つまみ	画面の名前	式
trim	altitude	Altitude	\$ALTITUDE

わかったこと

- ・ 式の入ったつまみは 488 個。大半はフレーム範囲（\$FSTART・\$FEND）。
- ・ set は式を消さない。値を入れたいときは deleteAllKeyframes() が先。
- ・ @N などの属性の式は要注意。入力に属性が無いと 0 になり、黙って効かない（ray が動かなかった）。

次にやること

- ・ DOP・LOP でも同じように数える
- ・ hou.Parm.set ではなく setExpression / 値の上書き（lock）の振る舞いを比べる

VEX のノイズの値の範囲 — noise() は 0.06~0.92 にしか届かず、snoise() は ±1 を超える

2026-09-24

何を作ったか

VEX のノイズ関数を、約100万点（100×100×100 の格子、間隔 0.37）で呼んで、出てきた値の最小・最大・平均・標準偏差を数えた。「0~1」「-1~1」という思い込みを実測で確かめる。

****noise() は 0~1 の真ん中に固まる。*** 最小 0.06132・最大 0.92433・平均 0.49996・標準偏差 0.10024。0 や 1 には近づかない。マスクに使うなら、fit で伸ばさないとコントラストが弱い。xnoise() も似ていて 0.04157~0.98223（標準偏差 0.12594）。

****snoise() は -1~1 に収まらない。*** -1.94708~2.27867。平均も 0.02992 と少しプラスに寄る。

onoise() は -0.94249~0.98991（平均ほぼ0）、anoise() は 0~0.98619 で平均 0.13305（0 の側に寄る）、curlnoise() の x 成分は -1.47739~1.4839。

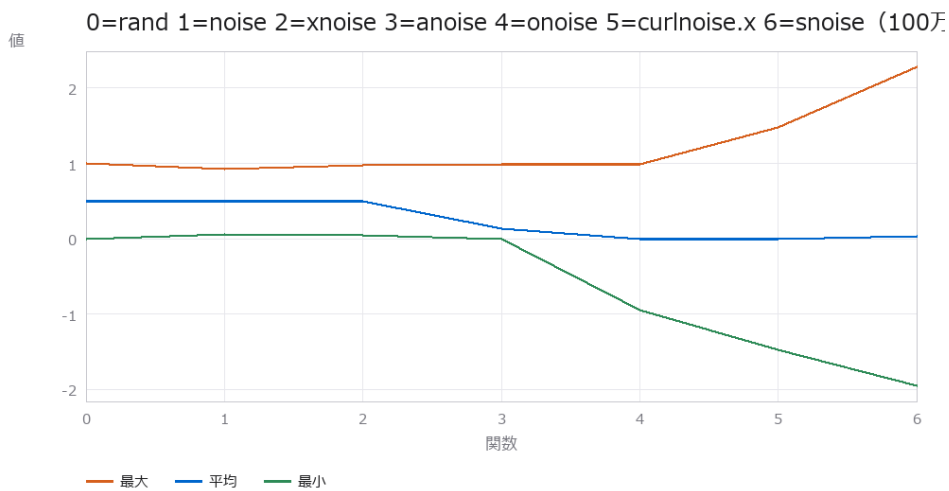
****flownoise(P, 0) は、noise(P)**

と同じ値だった******（最小・最大・標準偏差がすべて一致）。流れの位相が0のときは、ふつうの noise と同じ。vector を受ける noise() の x 成分も、float の noise() と同じ値だった。

比べとして rand() は 0~1、標準偏差 0.2888（一様なら 0.2887）。

関数ごとの値の範囲（約100万点）

位置は整数格子 × 0.37 + (0.123, 0.456, 0.789)。周波数は変えていない。秒は1回の計算（最初の関数はコンパイルを含む）。



noise・xnoise は 0.5 のまわりに狭く、snoise は ±1 を超える。

関数	最小	最大	平均	標準偏差	0~1 に入る割合	秒
noise(P)	0.06132	0.92433	0.49996	0.10024	100.0%	0.160
snoise(P)	-1.94708	2.27867	0.02992	0.34386	53.3%	0.033
onoise(P)	-0.94249	0.98991	-2e-05	0.27386	49.8%	0.029
anoise(P)	0	0.98619	0.13305	0.15766	100.0%	0.033

関数	最小	最大	平均	標準偏差	0~1 に入る割合	秒
xnoise(P)	0.04157	0.98223	0.50006	0.12594	100.0%	0.031
flownoise(P, 0)	0.06132	0.92433	0.49996	0.10024	100.0%	0.030
curlnoise(P).x	-1.47739	1.4839	-6e-05	0.34692	49.9%	0.033
vector noise(P).x	0.06132	0.92433	0.49996	0.10024	100.0%	0.031
rand(P)	0	1	0.5005	0.2888	100.0%	0.029

わかったこと

- noise() は 0.06~0.92。0~1 いっぱいに使いたいなら fit(v, 0.1, 0.9, 0, 1) などで伸ばす（端の値は目安。場所を変えれば少し変わる）。
- snoise() は ± 2 近くまで出る。clamp せずに色や高さに使うと、はみ出す。
- anoise() は 0 の側に寄る。平均 0.13。
- flownoise の位相0は noise と同じ。

次にやること

- 周波数を上げ下げしても、範囲は変わらないか
- unifiednoise の各種類で、Output を正規化したときの範囲
- オクターブを重ねた（フラクタル）ときの範囲の広がり

nearpoints は自分を先頭に数え、近い順に返す ー 半径ちょうど1は入るが、ちょうど√2は入らなかった

2026-09-24

何を作ったか

間隔1の格子（21×21）の真ん中の点で nearpoints(0, P, 半径, 最大数) を呼び、返ってきた点の数・順番・距離を調べた。格子なら、答えは数えられる（半径1で自分+上下左右の5、√2で斜めを足して9、2で13）。

自分自身も数に入り、いつも先頭に来る。
どの半径でも、1番目は距離0の自分。ほかの点は、距離の近い順に並んでいた。

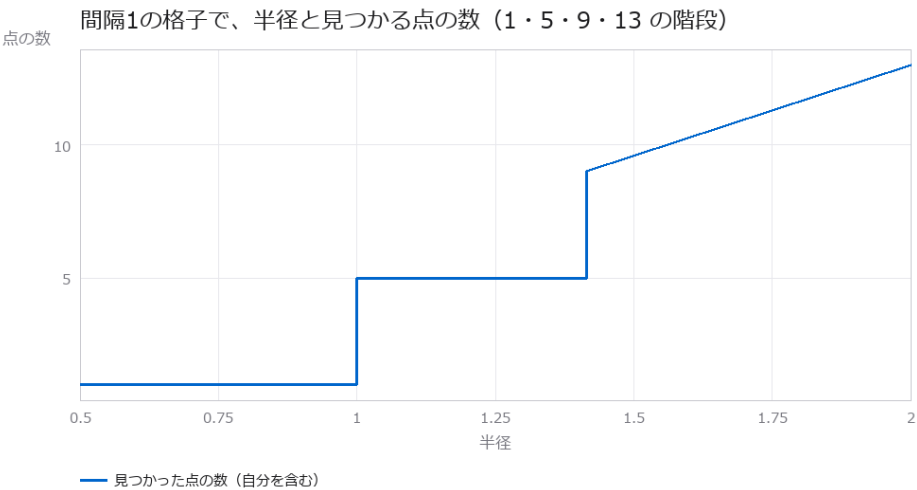
最大数は自分を含めて数える。 最大数3なら、自分+いちばん近い2点。1なら自分だけ。「近くの別の点を1つ」のつもりで最大数1にすると、自分しか返らない。

半径ちょうど 1.0 の点は入った（5点）。 ところが半径を √2 ちょうど（1.414214）にすると、斜めの点は入らず5点のまま。1.4143 にすると9点になった。√2 は小数で正確に表せないので、ほんのわずかな丸めで外に出る。境目の点を確実に入れたいなら、半径を少し大きくする。

pcfind() は、すべての場合で nearpoints() と同じ結果だった。

半径・最大数と、返ってきた点

距離は、返ってきた点までの距離を順に（最初の14個まで）。



半径が格子の距離（1・√2・2）を越えるたびに、点が増える。

関数	半径	最大数	点の数	自分が先頭	近い順	距離
nearpoints	0.5	100	1	はい	はい	0
pcfind	0.5	100	1	はい	はい	0
nearpoints	0.999	100	1	はい	はい	0
pcfind	0.999	100	1	はい	はい	0

関数	半径	最大数	点の数	自分が先頭	近い順	距離
nearpoints	1	100	5	はい	はい	0, 1, 1, 1, 1
pcfind	1	100	5	はい	はい	0, 1, 1, 1, 1
nearpoints	1.001	100	5	はい	はい	0, 1, 1, 1, 1
pcfind	1.001	100	5	はい	はい	0, 1, 1, 1, 1
nearpoints	1.41421	100	5	はい	はい	0, 1, 1, 1, 1
pcfind	1.41421	100	5	はい	はい	0, 1, 1, 1, 1
nearpoints	1.41431	100	9	はい	はい	0, 1, 1, 1, 1, 1.41421, 1.41421, 1.41421, 1.41421
pcfind	1.41431	100	9	はい	はい	0, 1, 1, 1, 1, 1.41421, 1.41421, 1.41421, 1.41421
nearpoints	2	100	13	はい	はい	0, 1, 1, 1, 1, 1.41421, 1.41421, 1.41421, 1.41421, 2, 2, 2, 2
pcfind	2	100	13	はい	はい	0, 1, 1, 1, 1, 1.41421, 1.41421, 1.41421, 1.41421, 2, 2, 2, 2
nearpoints	2	3	3	はい	はい	0, 1, 1
pcfind	2	3	3	はい	はい	0, 1, 1
nearpoints	2	1	1	はい	はい	0
pcfind	2	1	1	はい	はい	0

わかったこと

- ・ 自分も返る。ほかの点だけがほしいなら、最大数を1つ多くして先頭を捨てる。
- ・ 近い順に並ぶ。2番目が「いちばん近い別の点」。
- ・ 半径の境目は丸めで揺れる。格子の間隔ちょうどを半径にするなら、少し足しておく。

次にやること

- ・ 同じ距離の点どうしの並び順（点番号の順か）
- ・ 点が何百万あるときの速さと、最大数の効き目

VEX の fit は範囲の外で止まり、efit と lerp は伸び続ける — smooth は $3x^2-2x^3$

2026-09-24

何を作ったか

x を $-0.5 \sim 1.5$ まで振って、範囲を直す VEX の関数が、範囲の外でどうなるかを見た。

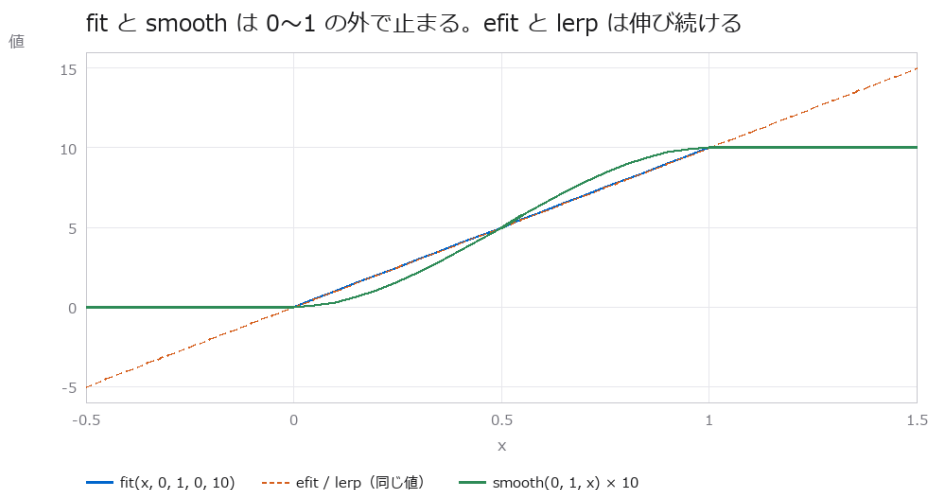
fit() と fit01() は、範囲の外で止まる（クランプする）。 fit(x, 0, 1, 0, 10) は x = -0.5 で 0、x = 1.5 で 10。

efit() と lerp() は、止まらずに伸びる。 同じ x で -5 と 15 。この2つは全部の x で同じ値だった。範囲の外も比例で伸ばしたい（外挿したい）ときは efit を使う。

smooth(0, 1, x) は、 $0 \sim 1$ の中では $3x^2 - 2x^3$ （スムーズステップ）と一致（ずれ最大 0.000000 ）し、外では 0 と 1 で止まる。両端でなめらかに止まる曲線。

clamp(x, 0, 1) は、もちろん 0 と 1 で止まる。

関数ごとの値 (x = $-0.5, 0, 0.5, 1, 1.5$)



fit (青) は 0 と 10 で止まり、efit・lerp (橙の破線) はまっすぐ伸びる。

関数	x=-0.5	x=0	x=0.5	x=1	x=1.5	範囲の外
fit(x, 0, 1, 0, 10)	0	0	5	10	10	止まる
efit(x, 0, 1, 0, 10)	-5	0	5	10	15	伸びる
fit01(x, 0, 10)	0	0	5	10	10	止まる
lerp(0.0, 10.0, x)	-5	0	5	10	15	伸びる
smooth(0.0, 1.0, x)	0	0	0.5	1	1	止まる
clamp(x, 0.0, 1.0)	0	0	0.5	1	1	止まる

わかったこと

- fit はクランプする。範囲の外の値を残したいなら efit。
- efit と lerp は同じ直線。書きやすい方を使えばよい。
- smooth は $3x^2 - 2x^3$ 。端でなめらかに止めたいマスクに向く。

次にやること

- fit の元の範囲を逆 (1, 0) にしたときの振る舞い
- chramp (ランプ) の範囲の外

122～149 の振り返り点検 —

28本すべて流し直し、時間以外の数字は全部一致

2026-09-24

何を作ったか

30件ごとの点検（030・060・090・121 に続いて5回目）。122～149 の28本を、121 と同じやり方で全部流し直した。記録（out/NNN_*.json）を控えに退避し、台本を hython で流し、新しい値と控えを数字1つずつ比べて、最後に控えを戻した（記録は変えていない）。

比べた値は 6,316 個。かかった時間は全部で 54 秒。

**時間以外の数字は全部一致した。 **時間の項目は流すたびに揺れるので、比べる対象から外している。

121

で見つかった「ファイルの大きさが1バイト違う」「記録に手で足した項目」の類いは、今回は無かった。

**記事を書く前に直したもの: 1件。 **実験149 で、Bricker

の線は原点から引かれると一度書いたが、Offset 0.05・Size 0.3 の枚数が予測と合わなかった。点の位置を測り直すと、線は形の端から引かれていた。公開前に記事を直した。予測の式を表に並べておいたので、ずれに気づけた。

**理由が分かっていないもの: 1件。 **実験147 の Taper の体積（倍率を平均した値と 0.45% 違う）。

スクリプトで見つけた落とし穴: 122～149 で6件（最初から式の入ったつまみは set

しても変わらない 133・134、attribblur は Pin Border が入っていると曲線では何も変わらない

139、mirror の継ぎ目は距離がつまみ未満でないととまらない 145、clip の Both

は切り口の点を共有したまま 146、tube の円錐は先の点が重なったまま 148、Bricker

の升目は形の端から 149）。

実験ごとの測り直し

時間の項目（sec など）は除いて比べた。許す差は 0.000001（相対）。

122～149 を全部流し直して、記録と比べた（数字1つずつ）

122 一致 65 値	123 一致 66 値	124 一致 88 値	125 一致 174 値	126 一致 41 値	127 一致 23 値
128 一致 124 値	129 一致 360 値	130 一致 354 値	131 一致 170 値	132 一致 72 値	133 一致 38 値
134 一致 2,022 値	135 一致 63 値	136 一致 194 値	137 一致 287 値	138 一致 436 値	139 一致 110 値
140 一致 84 値	141 一致 84 値	142 一致 77 値	143 一致 98 値	144 一致 67 値	145 一致 185 値
146 一致 288 値	147 一致 504 値	148 一致 108 値	149 一致 134 値		

28本とも、時間以外の数字がすべて一致。

実験	台本	比べた値	時間以外のずれ	秒
122	122_polyexpand.py	65	0	1.1

実験	台本	比べた値	時間以外のずれ	秒
123	123_shrinkwrap.py	66	0	1.4
124	124_ksystem.py	88	0	1.0
125	125_counts.py	174	0	1.1
126	126_cluster.py	41	0	1.3
127	127_shortest_path.py	23	0	1.1
128	128_extracttransform.py	124	0	1.0
129	129_curvature.py	360	0	1.4
130	130_cusp.py	354	0	1.1
131	131_bend.py	170	0	3.3
132	132_promote.py	72	0	1.1
133	133_ray.py	38	0	1.0
134	134_default_expr.py	2,022	0	9.9
135	135_noise_range.py	63	0	2.6
136	136_nearpoints.py	194	0	1.2
137	137_vex_fit.py	287	0	1.1
138	138_sdf_values.py	436	0	1.3
139	139_attribblur.py	110	0	1.5
140	140_copy_attrs.py	84	0	1.2
141	141_points_from_volume.py	84	0	2.8
142	142_platonic.py	77	0	1.0
143	143_circle.py	98	0	1.0
144	144_torus.py	67	0	1.6
145	145_mirror_seam.py	185	0	1.0
146	146_clip_cap.py	288	0	8.3
147	147_twist_ops.py	504	0	1.1
148	148_tube_cone.py	108	0	1.0
149	149_divide.py	134	0	1.0

わかったこと

- ・ 28本すべて流し直せた。全部で約54秒。
- ・ 予測の式を表に並べておくと、書き間違いに気づける。149 の Bricker がそうだった。
- ・ 次の点検は180。

次にやること

- ・ 実験147 の Taper の体積が平均と合わない理由

- ・時間の項目を3回の中央値で記録する

neighbourcount() の合計の半分は辺の数 — そこから出した $V - E + F$ は、板 1・球と箱 2・トーラスと筒 0

2026-09-24

何を作ったか

grid・box・sphere・torus・tube（蓋なし）に attribwrangle で $i@nb = neighbourcount(0, @ptnum)$ を入れ、隣の点の数を数えた。辺の数 E を「隣の数の合計 ÷ 2」で出し、面から数え直した辺の数とも比べた。

**隣の数の合計 ÷ 2 は、辺の数と5通りすべて一致。 **neighbourcount は「辺でつながった点」を数えている。

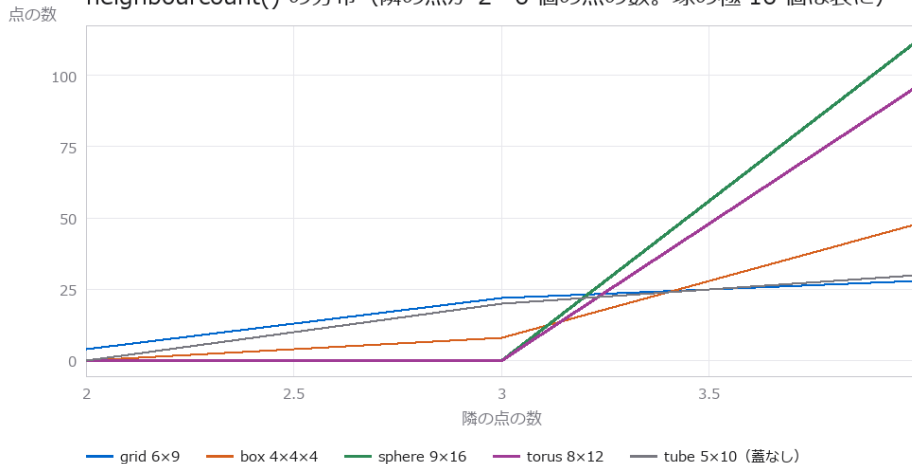
** $V - E + F$ （オイラー標数）は、板 1・球と箱 2・トーラスと蓋なしの筒 0 とすべて一致。 **形の穴や縁を、点と面の数だけで見分けられる。

**分布で形の場所が分かる。 **grid の角は 2、縁は 3、内側は 4。 box の角（8点）は 3。 sphere の極は Columns の数（16）だけ隣を持ち、残りは 4。 torus は全部 4。

**最初の1回だけ遅い。 **1つ目の wrangle は 0.0762 秒、2つ目からは 0.0013 秒以下。 VEX を最初に組み立てる時間とみられるが、確かめていない。

形ごとの点・辺・面と、 $V - E + F$

neighbourcount() の分布（隣の点が 2~6 個の点の数。球の極 16 個は表に）



grid だけ 2 と 3 が出る（角と縁）。torus は 4 だけ。

形	V (点)	Σ neighb ourcount	$E = \Sigma/2$	E (面から)	F (面)	$V - E +$ F	答え	分布（隣 の数: 点の数)	秒
grid 6x9	54	186	93	93	40	1	1	2: 4 / 3: 22 / 4: 28	0.0762
box 4x4x4	56	216	108	108	54	2	2	3: 8 / 4: 48	0.0012
sphere 9x16	114	480	240	240	128	2	2	4: 112 / 16: 2	0.0011

形	V (点)	Σ neighbourcount	$E = \Sigma/2$	E (面から)	F (面)	$V - E + F$	答え	分布 (隣の数: 点の数)	秒
torus 8×12	96	384	192	192	96	0	0	4: 96	0.0009
tube 5×10 (蓋なし)	50	180	90	90	40	0	0	3: 20 / 4: 30	0.0013

わかったこと

- ・ neighbourcount の合計 = 辺の数 × 2。辺の数を出す別の数え方として使える。
- ・ $V - E + F$ が 2 でなければ、閉じた1つの塊ではない。穴・縁・離れた塊の見つけ方になる。
- ・ 極の点は隣が多い。sphere の極は Columns 個。ぼかしや平滑化で極だけ効き方が変わる理由になる。

次にやること

- ・ neighbours() の並び順 (時計回りか)
- ・ fuse 前後で $V - E + F$ がどう変わるか

trail の速さは1秒あたり、差分の式どおり — 加速度は Central Difference のときだけ出て、Velocity Scale の2乗で縮む

2026-09-24

何を作ったか

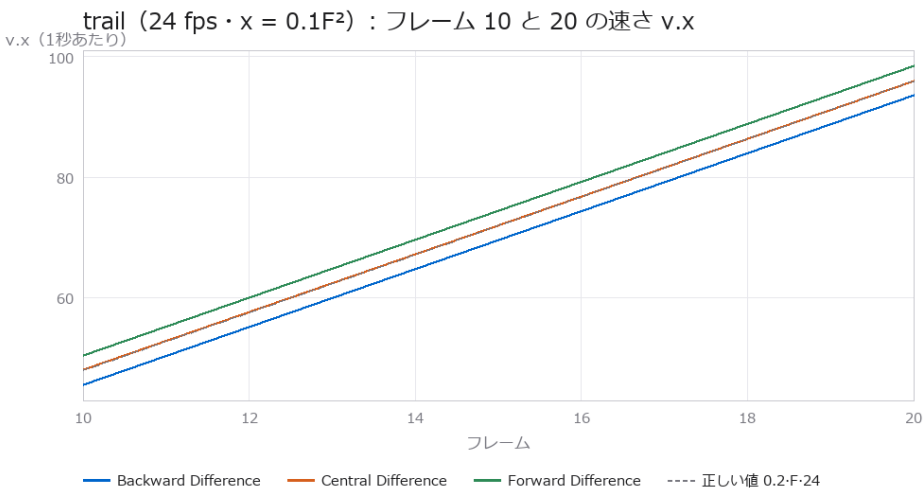
点を $x = 0.1 \cdot F^2$ (F はフレーム) で動かし、trail (Result Type = Compute Velocity) で v と加速度を出した。fps は 24 と 30、Velocity Approximation は3通り、Velocity Scale は 1 と 0.5。

**** v は1秒あたりで、差分の式と合った**** (24通りで差は最大 $5e-05$)。Backward は $(x(F) - x(F-1)) \cdot \text{fps}$ 、Forward は $(x(F+1) - x(F)) \cdot \text{fps}$ 、Central は2つの平均。 x が2次式なので Central だけが正しい値 $0.2 \cdot F \cdot \text{fps}$ にぴったり重なり、Backward は $0.1 \cdot \text{fps}$ 小さく、Forward は同じだけ大きい (24 fps・フレーム 10 で 45.6 / 48 / 50.4)。

****Compute Acceleration は Central Difference のときだけ値が入った。****Backward と Forward ではすべて 0。Central での値は $0.2 \cdot \text{fps}^2$ (24 fps で 115.2) と合った。

****Velocity Scale は v を掛け算し、加速度はその2乗で掛かる。****Scale 0.5 で v は半分、加速度は $1/4$ ($115.2 \rightarrow 28.8$ 。差は最大 0.0027)。時間の進みを縮めたのと同じ効き方。

fps・差分・Scale ごとの v と加速度



Central (橙) だけが点線に重なる。

fps	Velocity Approximation	Scale	フレーム	x	$v \cdot x$	差分の式	正しい値	加速度	正しい値	秒
24	Backward Difference	1	10	10	45.6000	45.6000	48	0.000	115.2	0.00023
24	Backward Difference	1	20	40	93.6000	93.6000	96	0.000	115.2	0.00006
24	Backward Difference	0.5	10	10	22.8000	22.8000	48	0.000	115.2	0.00006

fps	Velocity Approximation	Scale	フレーム	x	v.x	差分の式	正しい値	加速度	正しい値	秒
24	Backward Difference	0.5	20	40	46.8000	46.8000	96	0.000	115.2	0.00005
24	Central Difference	1	10	10	48.0000	48.0000	48	115.200	115.2	0.00008
24	Central Difference	1	20	40	96.0000	96.0000	96	115.198	115.2	0.00007
24	Central Difference	0.5	10	10	24.0000	24.0000	48	28.800	115.2	0.00010
24	Central Difference	0.5	20	40	48.0000	48.0000	96	28.800	115.2	0.00005
24	Forward Difference	1	10	10	50.4000	50.4000	48	0.000	115.2	0.00013
24	Forward Difference	1	20	40	98.4000	98.4000	96	0.000	115.2	0.00008
24	Forward Difference	0.5	10	10	25.2000	25.2000	48	0.000	115.2	0.00006
24	Forward Difference	0.5	20	40	49.2000	49.2000	96	0.000	115.2	0.00004
30	Backward Difference	1	10	10	57.0000	57.0000	60	0.000	180	0.00006
30	Backward Difference	1	20	40	117.0000	117.0000	120	0.000	180	0.00004
30	Backward Difference	0.5	10	10	28.5000	28.5000	60	0.000	180	0.00006
30	Backward Difference	0.5	20	40	58.5000	58.5000	120	0.000	180	0.00004
30	Central Difference	1	10	10	60.0000	60.0000	60	180.001	180	0.00007
30	Central Difference	1	20	40	120.0000	120.0000	120	179.997	180	0.00005
30	Central Difference	0.5	10	10	30.0000	30.0000	60	45.000	180	0.00007

fps	Velocity Approximation	Scale	フレーム	x	v.x	差分の式	正しい値	加速度	正しい値	秒
30	Central Difference	0.5	20	40	60.0000	60.0000	120	44.999	180	0.00006
30	Forward Difference	1	10	10	63.0000	63.0000	60	0.000	180	0.00006
30	Forward Difference	1	20	40	123.0000	123.0000	120	0.000	180	0.00004
30	Forward Difference	0.5	10	10	31.5000	31.5000	60	0.000	180	0.00005
30	Forward Difference	0.5	20	40	61.5000	61.5000	120	0.000	180	0.00004

わかったこと

- ・ v は 1 秒あたり。1 フレームあたりにしたいなら fps で割る。
- ・ 動きが加速しているなら Central Difference。Backward ・ Forward は半フレームずれた速さになる。
- ・ 加速度が 0 なら、差分が Central になっているか見る。

次にやること

- ・ Match by Attribute (id) で点の数が変わるときの v
- ・ Compute Angular Velocity の単位

curlnoise() はほぼ湧き出しのない流れ — 発散は、ふつうのノイズの約 3.5 万分の 1（周波数 1）

2026-09-24

何を作ったか

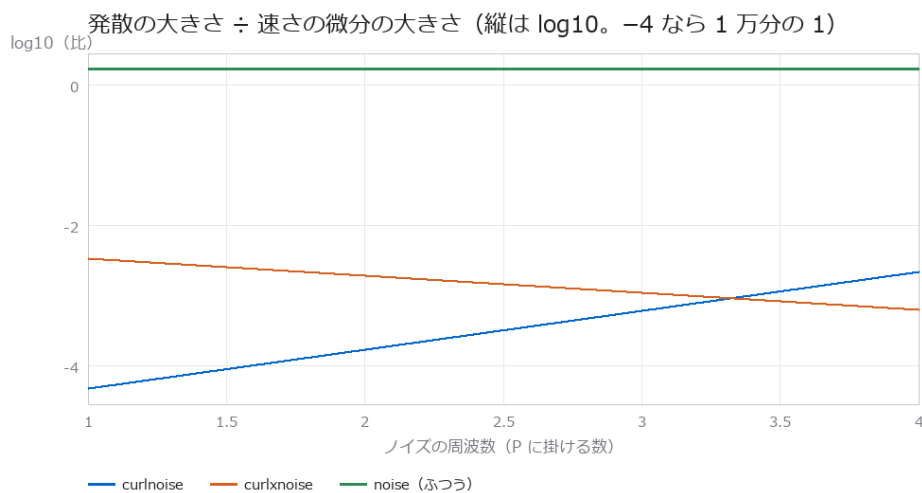
$1 \times 1 \times 1$ の中の $20^3 = 8,000$ 点で、VEX の $\text{curlnoise}(P \cdot f) \cdot \text{curlxnoise}(P \cdot f) \cdot \text{vector}(\text{noise}(P \cdot f))$ の値を中心差分 ($h = 0.001$) で微分し、発散 ($\partial v_x / \partial x + \partial v_y / \partial y + \partial v_z / \partial z$) を出した。比べやすいように、発散の平均の大きさを、速さの微分の大きさ ($|\partial v_x / \partial x|$ などの平均) で割った。

curlnoise の発散は、ほぼ 0。周波数 1 で比は $4.8e-05$ （1 万分の 0.5）、周波数 4 で $2.2e-03$ 。ふつうのベクトルノイズは 1.68 で、発散が速さの微分と同じくらいの大きさになる。周波数 1 で比べると、curlnoise の発散は 35,055 分の 1。

周波数を上げると、少し発散が出てくる。curlnoise は中でも微分を近似して回転を作っているとみられ、細かい模様ほどその近似の誤差が表に出る、と考えられるが、確かめていない（こちらの $h = 0.001$ の差分の誤差も含まれる）。

curlxnoise は平均では小さいが、ところどころ大きく跳ねる。周波数 1 で平均の比は $3.4e-03$ なのに、いちばん大きい発散は 2.21（curlnoise は $1.9e-04$ ）。模様の中に急に変わる所がある。

ノイズと発散



curlnoise (青) はふつうのノイズ (緑) より 3~4 桁小さい。

関数	周波数	点	発散 の平均	発散 の最大	速さの微分 の大きさ	比	速さの平均	秒
curlnoise	1	8,000	3.35e-05	1.94e-04	0.6984	4.79e-05	0.4987	0.0994
curlnoise	4	8,000	5.10e-03	7.58e-02	2.3425	2.18e-03	0.5464	0.0024
curlxnoise	1	8,000	3.24e-03	2.21e+00	0.9653	3.35e-03	0.8448	0.0025
curlxnoise	4	8,000	2.39e-03	1.73e+00	3.8452	6.21e-04	0.7675	0.0030

関数	周波数	点	発散 の平均	発散 の最大	速さの微分 の大きさ	比	速さの平均	秒
noise (ふ つう)	1	8,000	3.42e-01	1.01e+00	0.2036	1.68e+00	0.8486	0.0028
noise (ふ つう)	4	8,000	1.34e+00	6.48e+00	0.7923	1.69e+00	0.8689	0.0027

わかったこと

- ・粒を流して「かたまりのまま渦を巻く」動きにしたいなら curlnoise。湧き出し・吸い込みがほぼ無いので、粒は1点に集まらない（実験172で、混み具合がほぼ保たれるのを確かめた）。
- ・ふつうの noise を速さに使うと、湧き出しと吸い込みができる。粒が固まったり散ったりする。
- ・curlxnoise は、ところどころ発散が大きい。なめらかさが要るなら curlnoise。

次にやること

- ・curlnoise で流した粒の密度が本当に保たれるか
- ・curlnoise2d（2次元）の発散

同じ計算でも VEX（点ごと）は Detail のループの約170倍、Python の約260倍速い — 100 万点で 1.3 ミリ秒

2026-09-24

何を作ったか

grid の点に $@P.y = \sin(@P.x \cdot 10) \cdot \cos(@P.z \cdot 10) \cdot 0.1$ を入れる計算を4通りで書き、1 万・10 万・100 万点で時間をはかった（2〜3 回の一番速い値。1 回目の組み立ては先に済ませた）。

****VEX を点ごと（Run Over = Points）に流すのが圧倒的に速い。100 万点で 1.34 ミリ秒（1 点あたり 1.3**

ナノ秒）。点ごとに分けて、いくつもの計算を同時に進めているとみられる（確かめていない）。

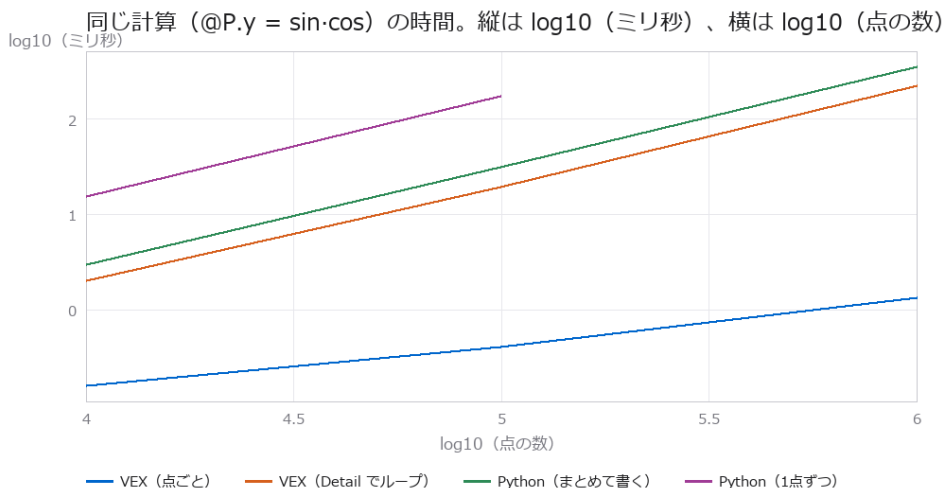
****同じ VEX でも、Detail で1回だけ走らせて for ループで回すと、100 万点で 222 ミリ秒（166 倍）。****ループが1本の流れで順に回り、同時に進められないためとみられる。

****Python は、値をまとめて読み書きしても 100 万点で 350 ミリ秒（VEX の 261 倍）。****点を1つずつ setPosition すると、さらに約 5.6 倍遅い（10 万点で 174 ミリ秒）。

****結果はどれも同じ****（y の差は最大 $2.0e-07$ 。Python は倍精度で計算して単精度に書くための差）。

****はかり方の注意:**** cook(force=True) だけでは VEX のノードが計算し直さず、どれも 0.00001 秒と出た。コードが読むつまみ (ch('k')) を毎回わずかに変えて、計算し直させてからはかった。

書き方・点の数と時間



VEX（点ごと・青）だけが桁違いに下。Detail のループ（橙）は Python（緑）と同じくらい。

点の数	書き方	秒	1 点あたり（マイクロ秒）	結果の差
10,000	VEX（点ごと）	0.00016	0.0163	0.0e+00
10,000	VEX（Detail でループ）	0.00203	0.2026	0.0e+00
10,000	Python（まとめて書く）	0.00298	0.2978	1.9e-07

点の数	書き方	秒	1 点あたり（マイクロ秒）	結果の差
10,000	Python（1点ずつ）	0.01553	1.5530	1.9e-07
99,856	VEX（点ごと）	0.00041	0.0041	0.0e+00
99,856	VEX（Detailでループ）	0.01920	0.1923	0.0e+00
99,856	Python（まとめて書く）	0.03133	0.3138	2.0e-07
99,856	Python（1点ずつ）	0.17413	1.7438	2.0e-07
1,000,000	VEX（点ごと）	0.00134	0.0013	0.0e+00
1,000,000	VEX（Detailでループ）	0.22212	0.2221	7.5e-09
1,000,000	Python（まとめて書く）	0.35007	0.3501	1.9e-07

わかったこと

- ・点ごとの計算は、Run Over = Points の wrangle で書く。Detail でループすると同時に進められず、約170倍遅い。
- ・Python で書くなら、値をまとめて読み書きする。1点ずつより約6倍速い。
- ・時間をはかるときは、本当に計算し直しているか確かめる。cook(force=True) だけでは VEX は計算し直さなかった。

次にやること

- ・Run Over = Primitives と Points の違い
- ・point() で他の点を読むときの遅さ

for-each はかけら1つあたり約 0.037 ミリ秒 — compile block で 35% 速く、wrangle 1つにまとめれば 30 倍速い

2026-09-24

何を作ったか

N 個の箱（name でかけらを分けた）を block_begin / block_end（Piece で name ごと）で1つずつ回し、中で「かけらの中心へ 0.8 倍に縮める」attribwrangle を1つだけ走らせた。compile_begin / compile_end で囲んだもの、そこに Multithread when Compiled を足したもの、繰り返しを使わず wrangle 1つで全部のかけらを処理したものと比べた（3 回の一番速い値）。

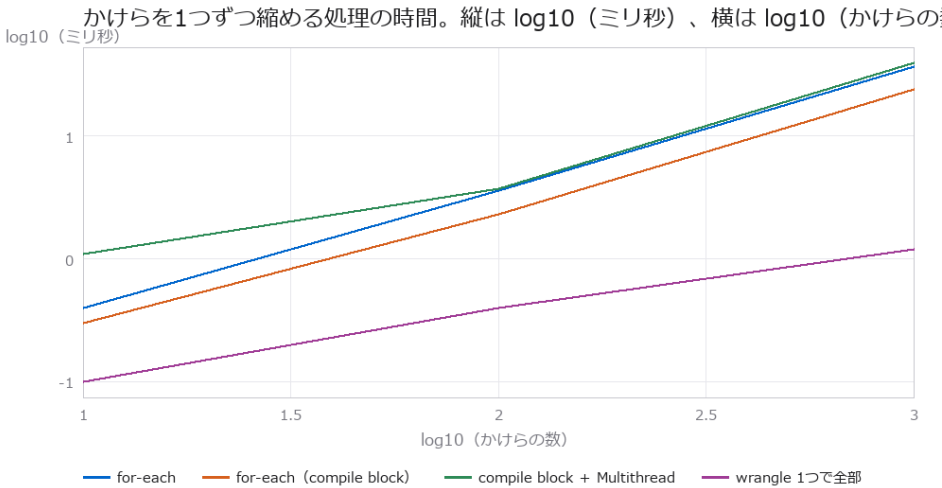
****結果の形は4通りとも同じ****（どの点も差 0）。

****for-each の時間は、かけらの数にほぼ比例する。****1 かけらあたり 10・100・1000 個で 0.039・0.036・0.037 ミリ秒。1000 個で 37 ミリ秒。

****compile block で約 35% 速くなった****（1000 個で 24 ミリ秒）。Multithread も入れると、かえって遅くなった（39 ミリ秒）。中の処理が軽すぎて、分けて配る手間の方が大きいとみられる。

****いちばん速いのは、繰り返しを使わず wrangle 1つで全部を処理すること。****1000 個で 1.2 ミリ秒（for-each の 31 分の 1）。かけらの中心は、同じ name の面の点から求めた。

かけらの数・書き方と時間



for-each の3本は傾き 1（数に比例）。wrangle 1つ（赤）は1桁以上下。

かけら	書き方	点	秒	1 かけらあたり（ミリ秒）	for-each との差
10	for-each	80	0.0004	0.0390	0
10	for-each (compile block)	80	0.0003	0.0276	0
10	compile block + Multithread	80	0.0011	0.1062	0

かけら	書き方	点	秒	1 かけらあたり (ミリ秒)	for-each との差
10	wrangle 1つで全部	80	0.0001	0.0081	0
100	for-each	800	0.0036	0.0362	0
100	for-each (compile block)	800	0.0023	0.0229	0
100	compile block + Multithread	800	0.0037	0.0367	0
100	wrangle 1つで全部	800	0.0004	0.0042	0
1000	for-each	8,000	0.0370	0.0370	0
1000	for-each (compile block)	8,000	0.0242	0.0242	0
1000	compile block + Multithread	8,000	0.0394	0.0394	0
1000	wrangle 1つで全部	8,000	0.0012	0.0012	0

わかったこと

- かけらごとの処理は、まず wrangle 1つで書けないか考える。1000 個で 30 倍速い。
- for-each を使うなら compile block で囲む。何もしなくても 35% 速くなる。
- Multithread は中の処理が重いときだけ。軽い処理ではかえって遅い。

次にやること

- 中の処理を重くしたとき (polyextrude など) の Multithread
- Fetch Feedback (繰り返しで形を積み上げる) の時間

pcfind の時間は「見つかった点の数」で決まる — 半径を広げても、最大の数 を 10 に絞れば 1 点 0.1 マイクロ秒

2026-09-24

何を作ったか

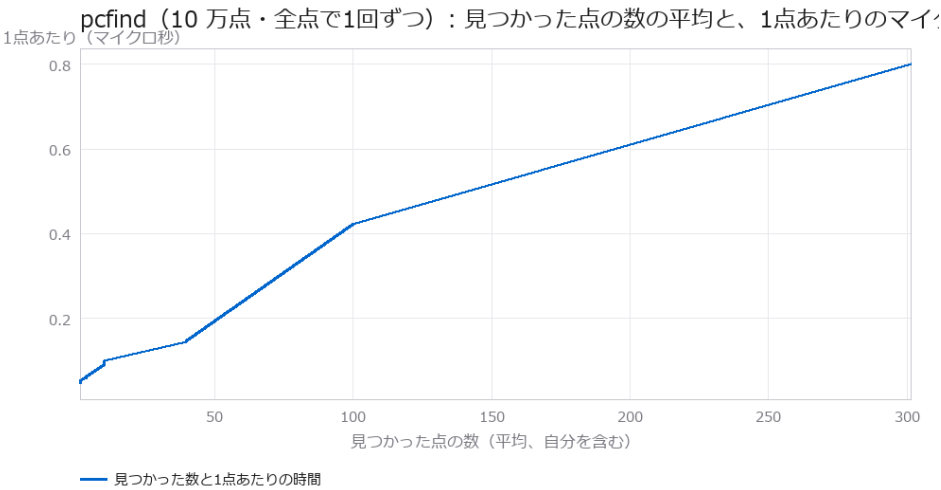
1 × 1 × 1 の中に一様に 10 万点を撒き、全部の点で pcfind(0, "P", @P, r, maxpts) を1回ずつ呼んだ。半径 r と最大の数 maxpts を変え、時間（2 回の速い方）と、見つかった数の平均を記録した。

****時間は、見つかった点の数にほぼ比例して増えた。***半径 0.1 で maxpts を 10・100・1000 にすると、見つかる数は 10・99.84・301.32、1 点あたり 0.10・0.42・0.80 マイクロ秒。半径の中にもっと多くの点があっても、maxpts で打ち切れば、打ち切った数の分しか時間がかからない。

****小さい半径では、ほとんど決まった手間だけ。***半径 0.01（見つかる数 1.32）で 1 点 0.05 マイクロ秒。10 万点で 5 ミリ秒。

****見つかる数には自分も入る。***半径 0.01 で、期待値（他の点 0.42 個）より多い 1.32 個になったのはそのため。半径 0.1 では、立方体の端の点が損をするので、期待値 418.88 より少ない 301.32 個。

半径・maxpts と時間



見つかった数（横）が増えるほど、ほぼ一直線に遅くなる。

半径	maxpts	見つかった数の平均	半径の中の期待値 (他の点)	秒 (10 万点)	1 点あたり (マイクロ秒)
0.01	10	1.32	0.42	0.0053	0.053
0.01	100	1.32	0.42	0.0047	0.047
0.01	1000	1.32	0.42	0.0050	0.050
0.02	10	3.53	3.35	0.0063	0.063
0.02	100	3.54	3.35	0.0057	0.057

半径	maxpts	見つかった数の平均	半径の中の期待値 (他の点)	秒 (10 万点)	1 点あたり (マイクロ秒)
0.02	1000	3.54	3.35	0.0061	0.061
0.05	10	10	52.36	0.0090	0.090
0.05	100	39.9	52.36	0.0145	0.145
0.05	1000	39.9	52.36	0.0147	0.147
0.1	10	10	418.88	0.0099	0.099
0.1	100	99.84	418.88	0.0423	0.423
0.1	1000	301.32	418.88	0.0800	0.800

わかったこと

- pcfind は maxpts で重さが決まる。要る数だけに絞る。
- 見つかる点には自分も入る。「まわりの数」なら 1 を引く（実験172 でそうした）。
- 10 万点・半径 0.05・100 点までで 15 ミリ秒。毎フレーム回しても軽い。

次にやること

- pcopen / pcfilter との速さの違い
- nearpoints() との速さの違い

151～179 の振り返り点検 —

29本すべて流し直し、時間以外の数字は全部一致

2026-09-24

何を作ったか

30件ごとの点検（030・060・090・121・150 に続いて6回目）。151～179 の29本を、150 と同じやり方で全部流し直した。記録（out/NNN_*.json）を控えに退避し、台本を hython で流し、新しい値と控えを数字1つずつ比べて、最後に控えを戻した（記録は変えていない）。

比べた値は 4,130 個。かかった時間は全部で 142 秒（Vellum の 170・177 が 24 秒と 51 秒）。

時間以外の数字は全部一致した。違ったのは時間の項目 34 個だけ（173・174・178 の「1点あたり」「1かけらあたり」の時間。流すたびに揺れる）。RBD・Vellum・POP・erode のようなシミュレーションも、同じ条件なら同じ数字を返した。

**公開前に直したもの: 5件。 **156

で折れ目の点の輪の選び方を2回まちがえた（点の番号の並びを思い込んだ。距離で選び直した）。172 で粒の混み具合を「升に分けて数える」方法で測ったが、群れの形の変化が混ざったので「まわりの粒の数」に替えた。173 で VEX の時間が 0.00001 秒と出た（cook(force=True) では計算し直さない。つまみを少し変えて計算し直させた）。175 で popwind の風のつまみ名をまちがえ、粒が動かなかった（windx が正しい）。170 の題が Substeps 1 のときしか成り立たない言い方だったので、条件を付けた。

**あとの実験で書き直したもの: 3件。 **157 の「relax で種を広げればそろはず」は、158 で逆（かえってばらつく）と分かったので直した。171 の「粒は1点に集まらないはず」は、172 で確かめて言い切りに直した。171 の「3 万分の 1」は計算した値（35,055）に合わせて「約 3.5 万分の 1」に直した。

**置き場所の直し: 1件。 **シミュレーションの実験7本（157～159・163・165～167）がモデリング編に入っていたので、エフェクト編へ移した。

スクリプトで見つけた落とし穴: 151～179 で8件（polyfill の四角形の塞ぎ方は奇数の穴を塞がない 154、scatter の relax で種が壁へ寄る 158、POP の粒は生まれたフレームで1ステップ先に進む 165、popdrag の終端速度は g/k でなく $\sqrt{g/k}$ 166、RBD の Collision Padding は止まる高さを変えない 167、heightfield_noise の Amplitude は高さの幅ではない 168、erode は土の量を保たない 169、cook(force=True) では VEX が計算し直さない 173）。

実験ごとの測り直し

時間の項目（sec・1点あたりの時間など）は除いて比べた。許す差は 0.000001（相対）。

151～179 を全部流し直して、記録と比べた（数字1つずつ）

151 一致 142 値	152 一致 180 値	153 一致 50 値	154 一致 169 値	155 一致 60 値	156 一致 89 値
157 一致 90 値	158 一致 135 値	159 一致 240 値	160 一致 241 値	161 一致 312 値	162 一致 112 値
163 一致 54 値	164 一致 109 値	165 一致 121 値	166 一致 209 値	167 一致 624 値	168 一致 120 値
169 一致 65 値	170 一致 57 値	171 一致 48 値	172 一致 24 値	173 一致 44 値	174 一致 84 値
175 一致 517 値	176 一致 96 値	177 一致 49 値	178 一致 60 値	179 一致 29 値	

29本とも、時間以外の数字がすべて一致。

実験	台本	比べた値	時間以外のずれ	秒
151	151_sphere_freq.py	142	0	1.2
152	152_bevel_volume.py	180	0	1.1
153	153_neighbours.py	50	0	1.1
154	154_polyfill_modes.py	169	0	1.0
155	155_scatter_density.py	60	0	4.5
156	156_polywire.py	89	0	1.0
157	157_voronoi_pieces.py	90	0	1.3
158	158_voronoi_relax.py	135	0	1.6
159	159_trail_velocity.py	240	0	1.0
160	160_uvlayout_fill.py	241	0	1.2
161	161_attribtransfer_blend.py	312	0	1.1
162	162_subdiv_crease.py	112	0	1.3
163	163_vellum_counts.py	54	0	1.7
164	164_vdb_particles.py	109	0	3.8
165	165_pop_gravity.py	121	0	2.1
166	166_pop_drag.py	209	0	11.8
167	167_rbd_rest.py	624	0	3.3
168	168_hf_noise_amp.py	120	0	1.8

実験	台本	比べた値	時間以外のずれ	秒
169	169_hf_erode_mass.py	65	0	4.1
170	170_vellum_stretch.py	57	0	23.8
171	171_curl_divergence.py	48	0	1.5
172	172_curl_advect.py	24	0	1.4
173	173_vex_vs_python.py	44	0	5.0
174	174_foreach_compile.py	84	0	1.9
175	175_pop_wind.py	517	0	3.9
176	176_hf_element_size.py	96	0	1.8
177	177_vellum_iterations.py	49	0	51.0
178	178_pcfind_cost.py	60	0	2.0
179	179_pop_wind_gravity.py	29	0	3.6

わかったこと

- ・29本すべて流し直せた。全部で約142秒。
- ・シミュレーションも、同じ条件なら同じ数字。比べる相手として使える。
- ・時間をはかる台本は、本当に計算し直しているかを最初に確かめる。173 で分かった。
- ・次の点検は210。

次にやること

- ・時間の項目を3回の中央値で記録する
- ・点検の台本を、範囲を引数で渡せるようにする

pcfind と nearpoints は同じ結果・同じ速さ — pcopen + pcfilter は 1.6 倍遅く、距離で重みを付けた平均になる

2026-09-24

何を作ったか

実験178 と同じ 10 万点で、全部の点について「半径 0.05 以内・最大 50 点」の位置の平均を、3通りの書き方で出した（2 回の速い方）。

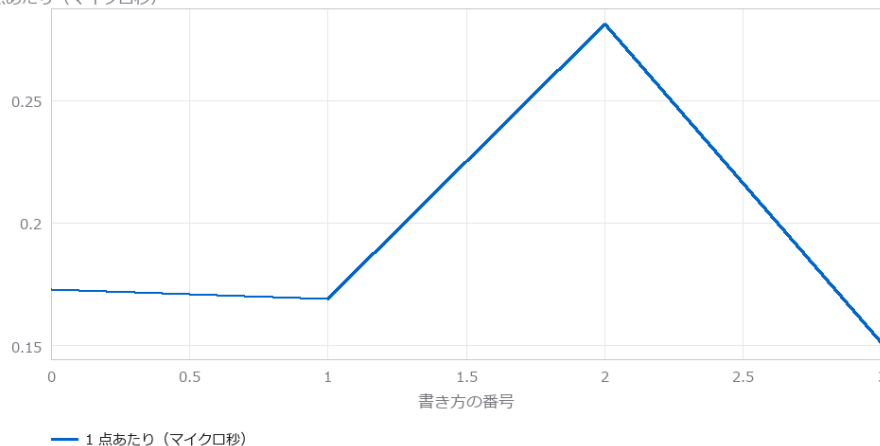
****pcfind と nearpoints は、結果が1点も違わなかった****（差 0）。時間も 17.3 と 16.9 ミリ秒でほぼ同じ。どちらも見つかる数は平均 38.41（端の点で少なくなる）。

****pcopen + pcfilter は 28.1 ミリ秒（1.6 倍）。****値も少し違い、平均の差は 0.0043、いちばん大きくて 0.0212。pcfilter は近い点ほど重く数える平均なので、ただの平均とはずれる。

****見つけた点の P を point() で読む手間は小さい。****数えるだけなら 15.1 ミリ秒、P を読んで平均すると 17.3 ミリ秒（+15%）。

書き方と、時間・結果

半径 0.05・最大 50 点の平均（10 万点）：0 pcfind / 1 nearpoints / 2 pcopen+pcfilter
1 点あたり（マイクロ秒）



2 (pcopen + pcfilter) だけが高い。

書き方	秒（10 万点）	1 点あたり（マイクロ秒）	見つけた数の平均	pcfind との差（平均）	差（最大）
pcfind + point	0.0173	0.173	38.41	0.00000	0.00000
nearpoints + point	0.0169	0.169	38.41	0.00000	0.00000
pcopen + pcfilter	0.0281	0.281	38.41	0.00427	0.02117
pcfind のみ（数えるだけ）	0.0151	0.151	38.41	（比べない）	—

わかったこと

・ 近くの点の番号が欲しいなら pcfind か nearpoints。どちらでも同じ。

- ・ pcfiler は重み付きの平均。ただの平均が欲しいなら、番号を取って自分で足す。
- ・ 見つけた点の値を読むのは安い。重いのは探すところ（実験178）。

次にやること

- ・ pcfiler の重みの形（距離との関係）
- ・ pcfind_radius（点ごとの半径）の速さ

pcfilter

の重みは「半径に対する距離」だけでは決まらない
— 見つかった点どうしの並びで変わり、式は分からなかった

2026-09-24

何を作ったか

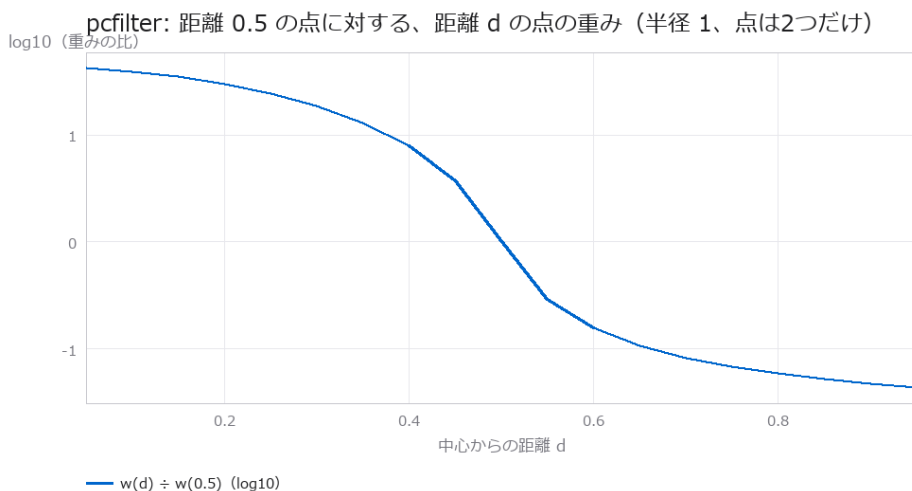
pcopen（半径 1）→ pcfiter で、点の重みが距離とともにどう下がるかを調べた。中心から 0.5 の所に $\text{val} = 0$ の点、距離 d に $\text{val} = 1$ の点を置き、結果 a から重みの比 $w(d)/w(0.5) = a/(1 - a)$ を出した。

**近い点ほど重い、形は単純な式にならない。 ** $d = 0.05$ で 41.9 倍、 $d = 0.95$ でも 0.044 倍（半径の端でも 0 にならない）。 $d = 0.5$ をはさんで急に変わり、両端では平らになる。

**3~4 点で候補の式と比べても、ぴったり合うものは無かった。 ** d を「いちばん遠く見つかった点の距離」で割った $1 - t$ がいちばん近かったが、それでも最大 0.027 ずれた ($1 - t/\text{farthest}$)。次が半径で割った $(1 - t^2)^2$ で、最大 0.037。重みは、見つかった点の並び（いちばん遠い点など）で決まっているとみられるが、確かめていない。

**中心ぴったりの点は特別に重い。 **最初に $\text{val} = 0$ の点を中心 ($d = 0$) に置くと、 B をどこに置いても結果が同じ 0.0228 になり、形が読めなかった。

2点での重みの比



$d = 0.5$ の前後で急に下がり、両端では平ら。

d	pcfilter の結果 a	$w(d) \div w(0.5)$
0.05	0.976709	41.9355
0.1	0.975120	39.1936
0.15	0.972297	35.0968
0.2	0.967775	30.0322
0.25	0.960610	24.3871

d	pcfilter の結果 a	$w(d) \div w(0.5)$
0.3	0.948845	18.5484
0.35	0.928074	12.9032
0.4	0.886861	7.8387
0.45	0.789116	3.7419
0.5	0.500000	1.0000
0.55	0.225689	0.2915
0.6	0.136120	0.1576
0.65	0.096975	0.1074
0.7	0.076132	0.0824
0.75	0.063534	0.0678
0.8	0.055237	0.0585
0.85	0.049425	0.0520
0.9	0.045162	0.0473
0.95	0.041923	0.0438

3～4 点で、候補の重みの式と比べる

点 (距離, val)	pcfilter の結果	1-t/farthest	$(1-t^2)/\text{radius}$	$(1-t^2)/\text{farthest}$	1-t/radius
(0.2, 1) / (0.5, 0) / (0.9, 0)	0.6342	0.6364	0.6062	0.6541	0.5714
(0.3, 1) / (0.6, 0) / (0.8, 0) / (0.95, 0)	0.5917	0.5652	0.6015	0.6451	0.5185
(0.1, 0) / (0.4, 1) / (0.7, 0)	0.3252	0.3333	0.3626	0.3210	0.3333

わかったこと

- ・ pcfiler

は「近いほど重い平均」だが、重みの形は点の並びしだい。決まった重みで混ぜたいなら、pcfind で番号を取り、自分で重みを掛けて足す。

- ・ 中心ぴったりに点があると、その点の値にほぼなる。自分の点を含めて探すときは注意。
- ・ ただの平均が欲しいなら pcfiler は使わない（実験181）。

次にやること

- ・ pcfiler の重みを、点を増やして形から推定する
- ・ pcfarthest（いちばん遠い点の距離）との関係

多角形の球のいちばん深いへこみは $\sin^2(\Delta/2)$ — 分割を倍にするたびに 1/4、平均はその 0.56 倍

2026-09-24

何を作ったか

半径 1 の本当の球の上に 2 万点を一様に撒き、VEX の xyzdist で多角形の球 (Polygon Mesh、Columns = 2 × (Rows − 1)) までの距離を測った。多角形は球の内側に入るので、距離は「その場所のへこみ」になる。

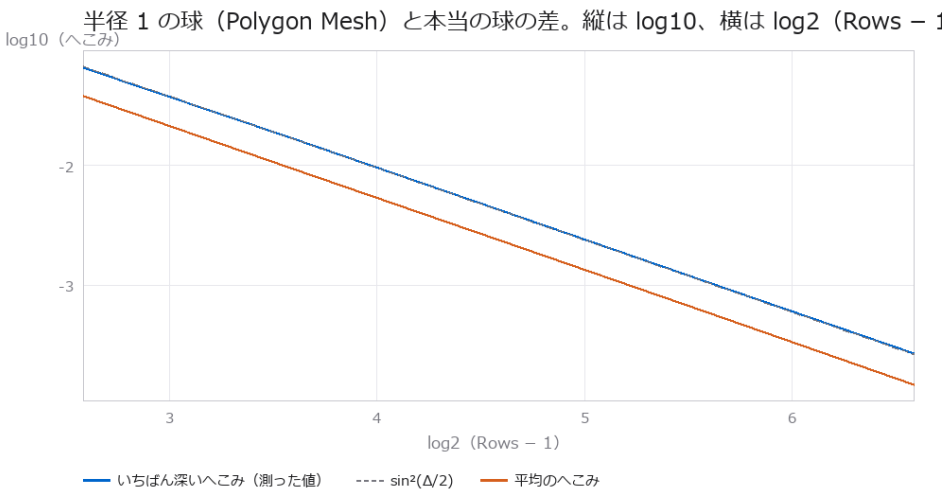
**いちばん深いへこみは、四角形の真ん中の $\sin^2(\Delta/2)$ に合った。 ** $\Delta = \pi / (\text{Rows} - 1)$ は隣の点どうしの角度。 Rows 97 で 0.0002676 (式 0.0002677) 、 Rows 13 で 0.01689 (式 0.01704) 。 辺の真ん中だけのへこみ $1 - \cos(\Delta/2)$ の、ほぼ 2 倍 (縦と横の両方向ぶん) 。

分割を倍にすると、へこみは約 1/4 (Rows を倍にするとに 3.84 → 3.96 → 3.99 → 4.0 分の 1) 。

平均のへこみは、いちばん深いへこみの約 0.56 倍 (0.576 · 0.562 · 0.558 · 0.557 · 0.556) 。

**xyzdist は速い。 **2 万点で 12 ミリ秒 (Rows 97、面 18,432 枚) 。

分割と、へこみ



測った値 (青) は点線の式にほぼ重なる。

Rows	Columns	いちばん深いへこみ	$\sin^2(\Delta/2)$	辺の真ん中 $1 - \cos(\Delta/2)$	平均	秒
7	12	0.0648701	0.0669873	0.0340742	0.0373800	0.0913
13	24	0.0168941	0.0170371	0.0085551	0.0094912	0.0031
25	48	0.0042660	0.0042776	0.0021411	0.0023806	0.0042
49	96	0.0010699	0.0010705	0.0005354	0.0005962	0.0055
97	192	0.0002676	0.0002677	0.0001339	0.0001488	0.0119

わかったこと

- ・ 球のへこみは $\sin^2(\pi/(2(\text{Rows} - 1)))$ 。欲しい精度から分割を逆算できる（へこみ 0.001 なら Rows 約 50）。
- ・ 分割を倍にすると誤差は 1/4、点の数は 4 倍。
- ・ 表面までの距離をはかるなら xyzdist。2 万点で数ミリ秒。

次にやること

- ・ Polygon（測地球）のへこみ
- ・ subdivide した球のへこみ